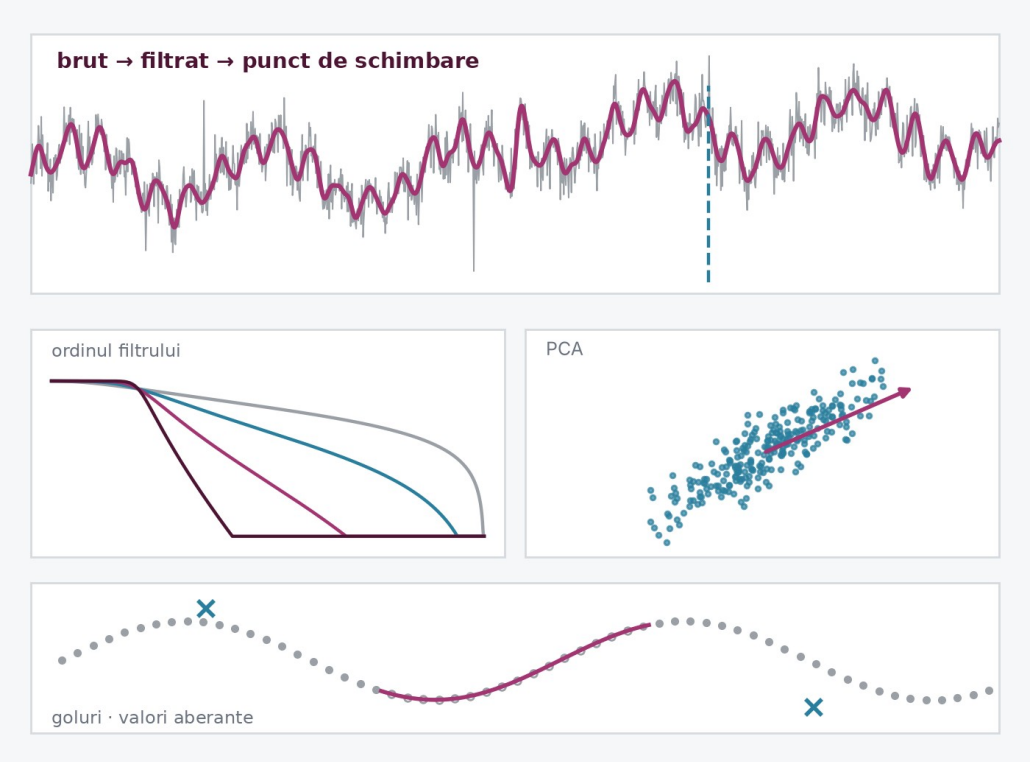
SISTEME BAZATE PE CUNOȘTINȚE

Lucrare de laborator · Anul universitar 2026–2027

Lucrare de laborator 4

Filtre, eliminarea tendinței, interpolare, valori aberante și puncte de schimbare

Pre-procesarea datelor II: cum alegem fiecare parametru al metodei – pe semnale sintetice în care adevărul este cunoscut și pe date reale în care nu este



CURS	DATE	CAIETE
Sisteme bazate pe cunoștințe	Semnale sintetice · puteri pe benzi EEG · trafic orar · birou inteligent	Exemple rezolvate + exerciții

Cuprins

1 Proiectarea unui filtru 6
- 1.1 Gândiți în Hz înainte să scrieți cod 6
- 1.2 Cele patru tipuri clasice de filtre 6
- 1.3 Media mobilă: un parametru, o regulă 7
- 1.4 Butterworth: frecvența de tăiere, ordinul și de ce folosim sos 8
- 1.5 Trece-bandă și notch 9
- 1.6 Cauzal sau cu fază zero? 10
- 1.7 FIR: mai mulți coeficienți, fără feedback, întârziere constantă 10
- 1.8 Netezire neliniară: median și Savitzky–Golay 11
- 1.9 Tabel de decizie 12
- 1.10 Date reale: trafic, EEG și CO_2 12

2 Proiectarea pasului de eliminare a tendinței 14
- 2.1 Ce fel de tendință și ce trebuie să rămână? 14
- 2.2 Eliminarea tendinței polinomiale: gradul 14
- 2.3 Media sau mediana mobilă: fereastra trebuie să fie multiplu al perioadei 15
- 2.4 Diferențierea: lag, ordin și ce costă 15
- 2.5 Trece-sus și eliminarea Fourier a tendinței: frecvența de tăiere 16
- 2.6 Staționară în jurul unei tendințe sau staționară prin diferențiere? 16
- 2.7 STL: tendință, parte sezonieră și rest împreună 17
- 2.8 Date reale: trafic și EEG 17

3 Proiectarea interpolării valorilor lipsă 19
- 3.1 Mai întâi priviți golurile 19
- 3.2 Metodele și parametrii lor 19
- 3.3 Depășirea și marcajele de timp neregulate 20
- 3.4 Evaluarea cinstită: mascați valori cunoscute, umpleți, măsurați 21
- 3.5 Imputare multivariată: KNN – și o greșeală clasică 21

4 Proiectarea unui detector de valori aberante 23
- 4.1 Defecțiuni, extreme și rolul cunoștințelor de domeniu 23
- 4.2 Reguli globale: z-score, MAD și IQR 23
- 4.3 Valori aberante într-o serie de timp: filtrul Hampel 24
- 4.4 Detectoare multivariate: Isolation Forest, LOF, DBSCAN 25
- 4.5 Ce facem după detecție 25
- 4.6 Date reale: biroul inteligent și traficul 25

5 Proiectarea reducerii dimensionalității 27
- 5.1 Extragere sau selecție? 27
- 5.2 PCA: întâi scalați, apoi alegeți numărul de componente 27
- 5.3 Selecția de trăsături: fiecare prag este un parametru 28
- 5.4 Reducere neliniară: t-SNE și autoencoderul 29
- 5.5 Date reale: puterile pe benzi EEG 29

6 Proiectarea unui detector de puncte de schimbare 31
- 6.1 Ce se schimbă și de ce am nevoie? 31
- 6.2 CUSUM: cel mai simplu detector și cei doi parametri ai lui 31
- 6.3 Detecție offline cu ruptures: cost, metodă de căutare și penalitate 32
- 6.4 Costul trebuie să se potrivească cu schimbarea 33
- 6.5 min_size, jump și marja de evaluare 33
- 6.6 Curățați mai întâi: valorile aberante creează puncte de schimbare false 33
- 6.7 Date reale: EEG și biroul 33

7 Un pipeline complet de pre-procesare 35
7.1 Ordinea pașilor 35
7.2 De ce eliminăm zgomotul după valorile aberante 35
7.3 O funcție de pipeline cu jurnal de decizii 35
7.4 Același pipeline pe datele de trafic 36

8 Rezumat și liste de verificare 37
8.1 Parametrii dintr-o privire 37
8.2 Întrebări înaintea fiecărui pas 37
8.3 Lecturi suplimentare 38

9 Tema 39
9.1 Caietul de exerciții 39
9.2 Mini-proiectul (Partea 7.3) 39
9.3 Predare 40

Acest laborator pe o pagină

DUPĂ ACEST LABORATOR VEȚI ȘTI SĂ

- **proiectați un filtru pornind de la o specificație**: alegeți tipul, frecvența de tăiere în Hz, ordinul sau lungimea și verificați răspunsul în frecvență;
- alegeți între filtrarea **cauzală și cea cu fază zero**, între **IIR și FIR**, între **median și Savitzky–Golay**;
- eliminați o **tendință** cu modelul potrivit (gradul polinomului, fereastra, lagul, frecvența de tăiere, STL) și să decideți între eliminarea tendinței și diferențiere cu testul ADF;
- **interpolați** golurile cu metoda potrivită lungimii golului și să o **validați** mascând valori cunoscute;
- stabiliți **pragul** unui detector de valori aberante (scor z, MAD, IQR, Hampel, Isolation Forest, DBSCAN) pornind de la valori aberante plantate;
- alegeți **numărul de componente** ale unui PCA, să evitați **scurgerea de informație** la selecție și să detectați **puncte de schimbare** cu o penalizare pe care o puteți apăra.

CUM FUNCȚIONEAZĂ LABORATORUL

- **Citiți** suportul: fiecare metodă are o *regulă de proiectare* pentru fiecare parametru.
- **Rulați** `Lab4_Exemple_Rezolvate.ipynb`: fiecare parametru este variat pe un semnal sintetic al cărui adevăr este cunoscut, apoi folosit pe date reale.
- **Rezolvați** `Lab4_Exercitii.ipynb`: 51 de exerciții; fiecare vă cere să *alegeți* un parametru pe baza unor dovezi. Autoverificările afișează ✅ sau ❌ și un indiciu.
- Scrieți **interpretarea** fiecărui exercițiu: de ce este bună această valoare?
- Încheiați cu **mini-proiectul** (Partea 7): un pipeline documentat pe date de trafic sau EEG.

Fișierele acestui laborator

Fișier	Ce este
`Lab4_Exemple_Rezolvate.ipynb`	Întregul conținut al acestui suport sub formă de cod rulabil: 7 părți, figuri, parcurgeri de parametri.
`Lab4_Exercitii.ipynb`	51 de exerciții cu autoverificări automate (Părțile 1–7) și mini-proiectul deschis.
`sleepy_driver_eeg_data.csv`	Puteri pe benzi EEG ale unui șofer (1 eșantion pe secundă, 3 735 de rânduri, etichetă de somnolență).
`Metro_Interstate_Traffic_Volume.csv`	Volum orar de trafic și vreme pe o autostradă, 2012–2018.
`smart_office_dataset.csv`	O săptămână de senzori de birou la 1 minut (temperatură, umiditate, CO_2, ocupare) cu defecțiuni injectate.

POVESTEA

Laboratorul 3 v-a dat un *diagnostic*: reguli de calitate, tendință și sezonalitate, spectrul. Cursurile 4–6 v-au dat *tratamentele*. Un tratament are parametri – o frecvență de tăiere, o fereastră, o penalizare, un prag – iar o valoare greșită face mai mult rău decât lipsa tratamentului: o fereastră care nu este multiplu al perioadei lasă o ondulație, un prag de 2σ marchează 5 % din date perfect bune, un spline depășește o treaptă. Acest laborator vă învață să **alegeți fiecare parametru pe baza unor dovezi**: mai întâi pe un semnal sintetic în care adevărul este cunoscut, ca să vedeți ce face parametrul, apoi pe date reale unde doar raționamentul vă protejează.

1 Proiectarea unui filtru

1.1 Gândiți în Hz înainte să scrieți cod

Un filtru păstrează unele frecvențe și le atenuează pe altele. Înainte de a alege o funcție, răspundeți la trei întrebări **în Hz**: *ce vreau să păstrez?* (ritmul alfa 8–12 Hz, ciclul zilnic 1/24 cicluri pe oră), *ce vreau să elimin?* (rețeaua de 50 Hz, o derivă sub 0.5 Hz, zgomot peste 40 Hz) și *cu ce frecvență este eșantionat semnalul?* Frecvența de eșantionare `fs` fixează cea mai mare frecvență utilizabilă, `fs / 2` (Nyquist). Răspunsurile dau **tipul** (trece-jos, trece-sus, trece-bandă, oprește-bandă) și **frecvențele de tăiere**; abia apoi alegem **familia** și **ordinul**.

Semnalul nostru sintetic de test are `fs = 250 Hz` și conține ce furnizează un amplificator EEG real: un ritm alfa de 10 Hz (dorit), o derivă de 0.3 Hz, interferența rețelei de 50 Hz, zgomot alb și trei vârfuri de electrod (artefacte de un singur eșantion).

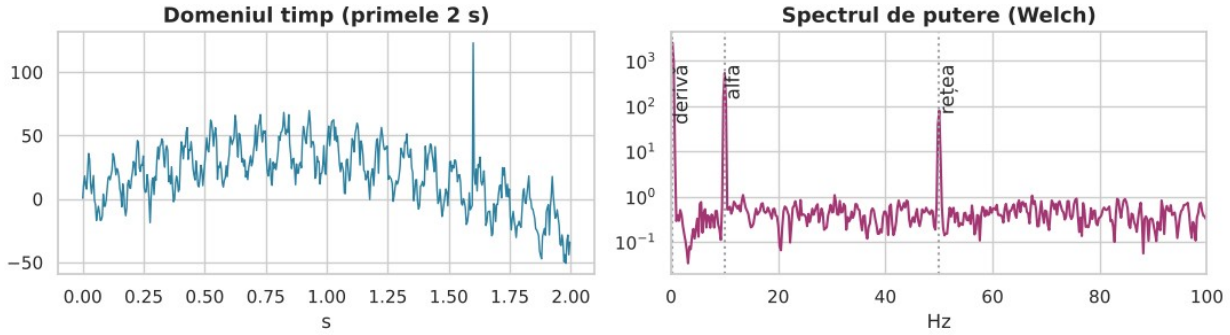


Figura 1. Semnalul sintetic de tip EEG. Banda dorită se află între derivă și rețea: un trece-bandă 5–30 Hz le elimină pe amândouă

> **CE ÎNVĂȚĂM**
> Spectrul este primul pas al **proiectării**, nu al analizei: trei vârfuri și un prag plat de zgomot vă spun tipul filtrului. Un trece-sus singur ar lăsa rețeaua, un notch singur ar lăsa deriva.

1.2 Cele patru tipuri clasice de filtre

Orice filtru din acest laborator este de **unul dintre cele patru tipuri**, definite de partea din spectru pe care o lasă să treacă. Partea păstrată este **banda de trecere**, partea respinsă este **banda de oprire**, iar între ele se află **banda de tranziție**, în care câștigul scade de la „păstrat” la „respins”. Un filtru real nu poate avea o margine verticală: cu cât cereți o bandă de tranziție mai îngustă, cu atât plătiți un **ordin mai mare** (sau un filtru FIR mai lung).

Tip	Păstrează (banda de trecere)	Respinge (banda de oprire)	Frecvențe de tăiere	Utilizare tipică
Trece-jos	sub `fc`	peste `fc`	una: `fc`	netezire, eliminarea zgomotului, anti-aliasing, componenta lentă a unei serii
Trece-sus	peste `fc`	sub `fc`	una: `fc`	eliminarea derivei sau a liniei de bază (eliminarea tendinței, Capitolul 2)
Trece-bandă	între `f1` și `f2`	sub `f1` și peste `f2`	două: `[f1, f2]`	izolarea unui ritm: alfa 8–12 Hz, ciclul zilnic al traficului
Oprește-bandă (notch)	tot, în afară de `[f1, f2]`	între `f1` și `f2`	două: `[f1, f2]`	eliminarea unei interferențe: rețeaua de 50 Hz

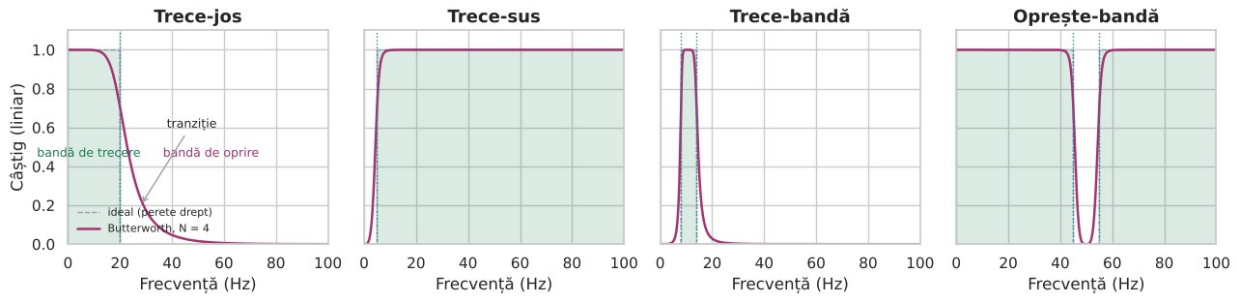


Figura 2. Cele patru tipuri clasice: răspunsul ideal „perete drept” (întrerupt, zona verde = banda de trecere) și un filtru Butterworth real de ordin 4 (roz). Liniile punctate sunt frecvențele de tăiere; colțul rotunjit este banda de tranziție

Frecvența de tăiere `fc` este locul unde câștigul a scăzut cu 3 dB (la 71 % din amplitudine); pentru un filtru Butterworth este mijlocul benzii de tranziție. În `buttord(wp, ws, gpass, gstop)` specificația este dată prin două margini de bandă: `wp` este marginea **benzii de trecere**, `ws` marginea **benzii de oprire**, `gpass` pierderea maximă tolerată în banda de trecere (dB) și `gstop` atenuarea minimă cerută în banda de oprire (dB). Ordinea celor două margini spune tipul: trece-jos `wp < ws`, trece-sus `wp > ws`; pentru tipurile de bandă ambele sunt liste cu două elemente, iar la un trece-bandă banda de trecere se află *în interiorul* marginilor benzii de oprire, la un oprește-bandă invers.

Aplicate semnalului nostru – o derivă de 0.3 Hz, un ritm alfa de 10 Hz și rețeaua de 50 Hz – cele patru tipuri se comportă astfel (câștig în dB la fiecare componentă; valorile apropiate de 0 înseamnă „păstrat”):

REZULTAT

```
                        derivă 0.3 Hz  alfa 10 Hz  rețea 50 Hz
trece-jos  fc = 20              0.0        -0.0        -36.1
trece-sus fc = 2              -65.9        -0.0         -0.0
trece-bandă [5, 30]          -103.8        -0.0        -25.4
oprește-bandă [45, 55]          0.0        -0.0       -135.2
```

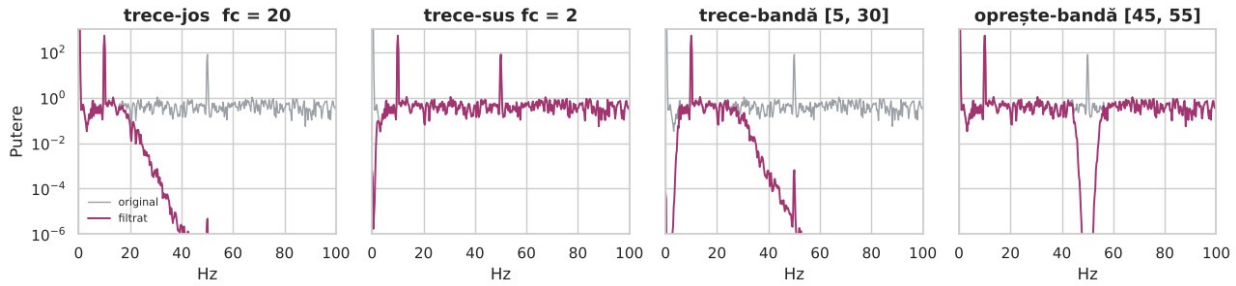


Figura 3. Spectrul de putere înainte (gri) și după (roz) fiecare filtru: tipul rezultă din ce se află de o parte și de alta a benzii dorite

CE ÎNVĂȚĂM

Cum alegeți tipul: desenați spectrul și întrebați ce se află *sub*, *între* și *deasupra* frecvențelor dorite. Nimic dorit deasupra → trece-jos; nimic dorit dedesubt → trece-sus; nedorit de ambele părți → trece-bandă; o singură linie îngustă nedorită → oprește-bandă. Abia apoi alegeți frecvențele de tăiere, familia și ordinul.

1.3 Media mobilă: un parametru, o regulă

O medie mobilă pe `w` eșantioane este un filtru trece-jos al cărui **prim zero se află la `fs / w`**. Regula de proiectare este deci: *alegeți `w` astfel încât primul zero să cadă pe frecvența pe care vreți să o eliminați.* Pentru rețeaua de 50 Hz la `fs = 250`, `w = 250 / 50 = 5`: media pe exact o perioadă a rețelei este zero. O fereastră de 6 nu ar elimina-o. Media mobilă este ieftină și nu produce depășiri, dar este un filtru slab: atenuare lentă și lobi laterali.

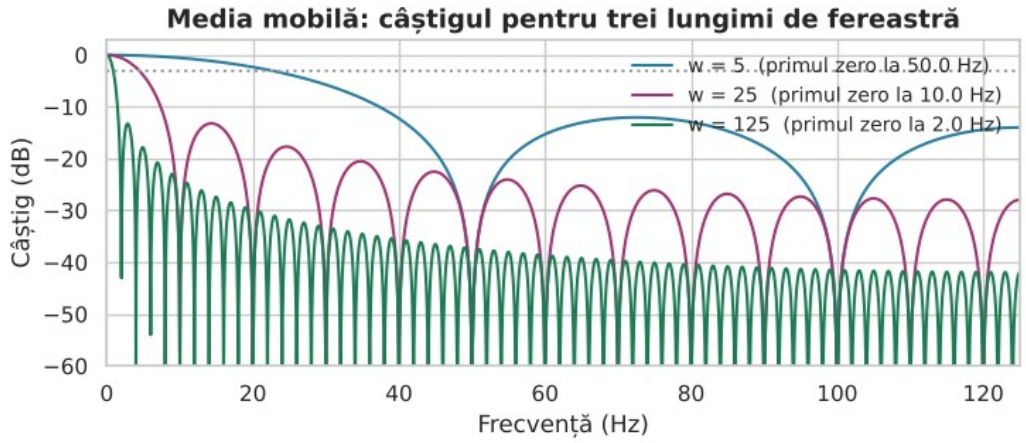


Figura 4. Câștigul mediei mobile pentru trei lungimi de fereastră: primul zero se află la fs / w

```
w = int(fs / 50)                                    # primul zero exact la 50 Hz
x_ma = np.convolve(x, np.ones(w) / w, mode="same")
```

REZULTAT

```
fereastră: 5 eșantioane
câștig la 10 Hz (alfa): -0.56 dB
câștig la 50 Hz (rețea): sub -100 dB (zero exact al răspunsului)
```

1.4 Butterworth: frecvența de tăiere, ordinul și de ce folosim sos

Filtrul Butterworth are banda de trecere cea mai plată posibilă. Are doi parametri. **Frecvența de tăiere** `fc` este locul unde câștigul este −3 dB; pentru un trece-bandă sau oprește-bandă se dă `[f_low, f_high]`. **Ordinul** `N` fixează abruptul: atenuarea crește cu aproximativ `20·N` dB pe decadă, dar fiecare pas costă și întârziere și risc numeric. Ca să le alegeți, puneți `fc` **între** frecvența dorită și cea nedorită, apoi luați **cel mai mic `N`** care dă atenuarea necesară. SciPy îl calculează pentru voi:

```
N, Wn = signal.buttord(wp=15, ws=40, gpass=1, gstop=40, fs=fs)
sos = signal.butter(N, Wn, btype="low", fs=fs, output="sos")
```

REZULTAT

```
cel mai mic ordin: 5   frecvență de tăiere: 17.11 Hz
```

Aici `wp` este sfârșitul benzii de trecere, `ws` începutul benzii de oprire, `gpass` pierderea maximă permisă în banda de trecere (dB) și `gstop` atenuarea minimă în banda de oprire (dB). Dați întotdeauna `fs=fs` (atunci frecvențele de tăiere sunt în Hz) și `output="sos"` (secțiuni de ordinul doi).

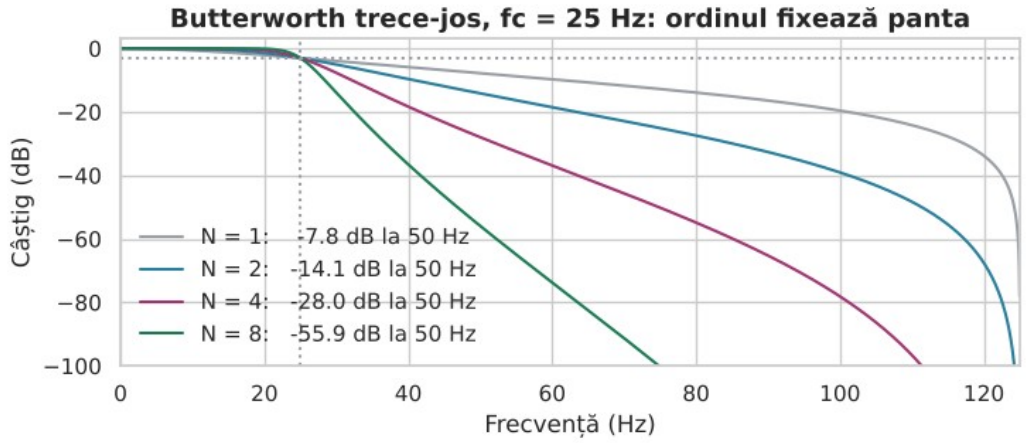


Figura 5. Butterworth trece-jos cu fc = 25 Hz: ordinul fixează panta; la 50 Hz fiecare dublare a lui N dublează aproximativ atenuarea în dB

DE CE NU `B, A`?

Forma polinomială pierde precizie când frecvența de tăiere este foarte mică față de `fs` sau ordinul este mare. Un trece-sus de 0.1 Hz de ordin 8 la `fs = 250 Hz` are un pol în afara cercului unitate în forma `b, a`: filtrul este **instabil**, iar ieșirea lui explodează. Forma `sos` a aceluiași filtru este stabilă.

REZULTAT

```
cea mai mare rază a polilor, forma b/a: 1.0146 (>1 înseamnă instabil)
max |y|  forma b/a: 1.69e+16    forma sos: 174
```

1.5 Trece-bandă și notch

Pentru ritmul alfa (8–12 Hz) folosiți banda de interes **cu o marjă** și ordinul 4: o bandă foarte îngustă cere un ordin mare și este sensibilă la frecvența exactă. Pentru **o singură** frecvență perturbatoare este mai bun un filtru notch. Parametrul lui este factorul de calitate **`Q = f0 / lățimea de bandă`**: cu `Q = 30` notch-ul de 50 Hz are 1.7 Hz lățime; cu `Q = 5` are 10 Hz și începe să consume semnalul din jur.

```
sos_bp = signal.butter(4, [5, 30], btype="bandpass", fs=fs, output="sos")
b, a = signal.iirnotch(50, Q=30, fs=fs)
```

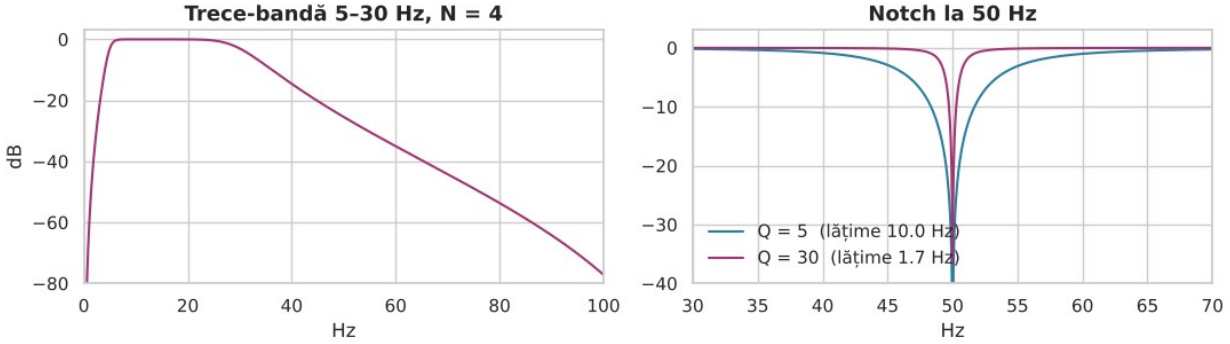


Figura 6. Stânga: trece-bandă 5–30 Hz, ordin 4. Dreapta: notch la 50 Hz pentru Q = 5 și Q = 30

1.6 Cauzal sau cu fază zero?

Un filtru IIR întârzie semnalul, iar frecvențele diferite cu cantități diferite (**distorsiune de fază**). Dacă filtrați **offline**, rulați filtrul înainte și înapoi (`sosfiltfilt`): întârziere zero, dar ordinul efectiv se dublează, câștigul la `fc` devine −6 dB, iar marginile arată tranzitorii de pornire. Dacă filtrați **în timp real** – un sistem de monitorizare a șoferului nu se poate uita în viitor – este permis doar `sosfilt`, iar întârzierea trebuie acceptată și luată în calcul.

REZULTAT

```
întârziere cauzal    : 7 eșantioane = 28.0 ms
întârziere fază zero: 0 eșantioane
```

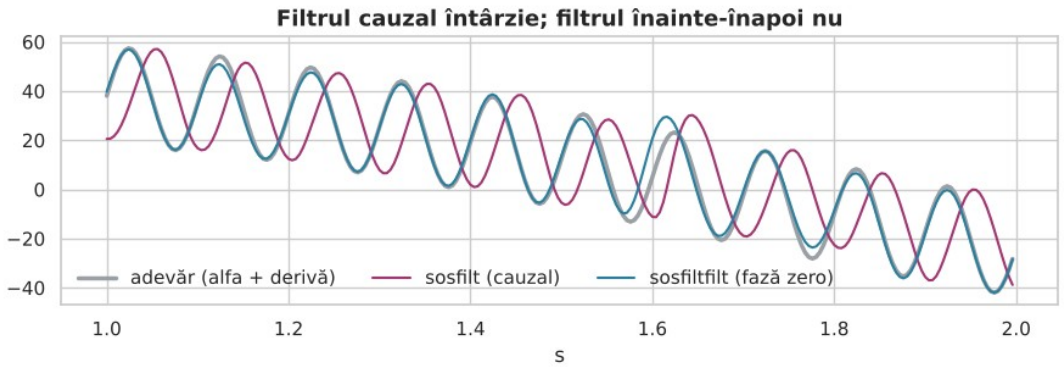


Figura 7. Un trece-jos de ordin 4 la 15 Hz: ieșirea cauzală întârzie cu 7 eșantioane (28 ms); cea cu fază zero nu

1.7 FIR: mai mulți coeficienți, fără feedback, întârziere constantă

Un filtru FIR este o medie mobilă ponderată cu `numtaps` coeficienți. Este întotdeauna stabil și are **fază liniară**: întârzierea este constanta `(numtaps − 1) / 2` eșantioane pentru orice frecvență. Prețul este lungimea. Lățimea tranziției este invers proporțională cu `numtaps` (pentru o fereastră Hamming `numtaps ≈ 3.3·fs / lățime`). Metoda Kaiser transformă o atenuare și o lățime într-o lungime:

```
numtaps, beta = signal.kaiserord(ripple=40, width=25 / (fs / 2))
numtaps += (numtaps % 2 == 0)                    # lungime impară: întârziere întreagă
h = signal.firwin(numtaps, 27.5, window=("kaiser", beta), fs=fs)
```

REZULTAT

```
coeficienți: 25   întârziere: 12 eșantioane = 48.0 ms
```

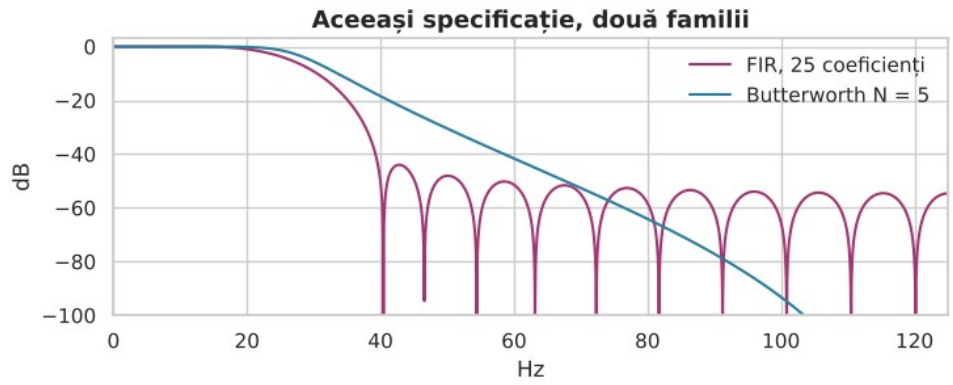


Figura 8. Aceeași specificație îndeplinită de un filtru FIR cu 25 de coeficienți și de un Butterworth de ordin 5

IIR atinge specificația cu o mână de coeficienți, dar are fază neliniară. FIR cere de câteva ori mai mulți coeficienți, dar are aceeași întârziere pentru toate frecvențele: este alegerea potrivită când contează **forma** undei (evenimente ECG sau EEG).

1.8 Netezire neliniară: median și Savitzky–Golay

Filtrele liniare împrăștie un vârf peste vecinii săi. Când problema sunt **vârfurile**, folosiți un **filtru median**: parametrul este `kernel_size` (impar), iar regula spune că nucleul trebuie să fie **mai mare decât dublul lățimii vârfului**. Când problema este **zgomotul pe un maxim neted**, folosiți **Savitzky–Golay**, care ajustează un polinom de grad `polyorder` într-o fereastră de `window_length` eșantioane: fereastra ar trebui să fie cam cât lățimea celei mai înguste trăsături pe care vreți să o păstrați, `polyorder` este 2–4 (mai mare urmărește datele mai îndeaproape și netezește mai puțin).

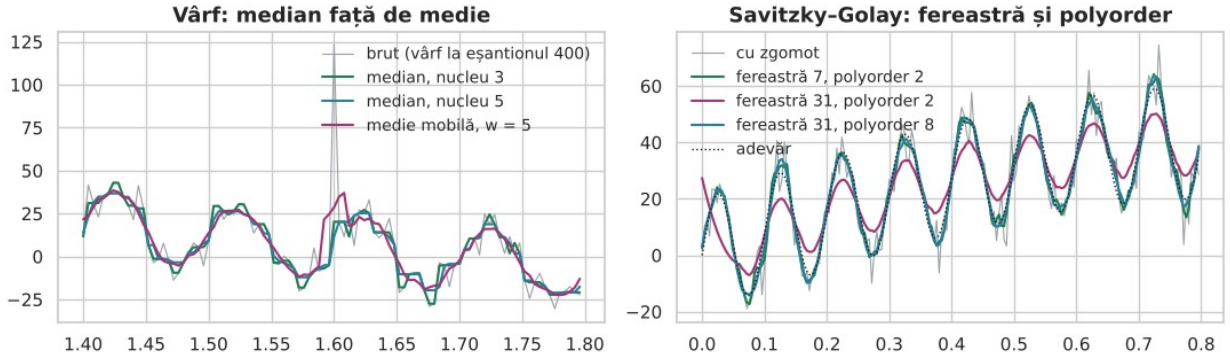


Figura 9. Stânga: un vârf de un eșantion; medianul îl elimină, media mobilă îl întinde. Dreapta: Savitzky–Golay cu o fereastră scurtă, una lungă și una lungă cu un ordin polinomial mare

1.9 Tabel de decizie

Problemă	Filtru	Parametru	Cum se alege
Zgomot peste banda de interes	Butterworth trece-jos	`fc`, `N`	`fc` între frecvența dorită și cea nedorită; `N` din `buttord`
Derivă / variația liniei de bază	Butterworth trece-sus	`fc`, `N`	`fc` ≈ ⅓ din cea mai joasă frecvență dorită; `N` = 2–4
Interferență la o singură frecvență	Notch (`iirnotch`)	`f0`, `Q`	`Q = f0 / lățimea de bandă`; îngust dacă vecinii contează
Perioadă cunoscută care trebuie mediată	Medie mobilă	`w`	`w = fs / f_interferență` sau un multiplu al perioadei
Vârfuri izolate	Median	`kernel_size`	impar, mai mare decât 2 × lățimea vârfului
Zgomot pe maxime netede	Savitzky–Golay	`window_length`, `polyorder`	fereastra ≈ cea mai îngustă trăsătură; `polyorder` 2–4
Forma undei contează, offline	FIR, fază liniară	`numtaps`	`numtaps ≈ 3.3·fs / lățimea tranziției`
Timp real	doar cauzal (`sosfilt`)	—	acceptați întârzierea

1.10 Date reale: trafic, EEG și CO_2

Traficul. Fișierul are eșantioane orare (`fs = 1` pe oră; Nyquist 0.5 cicluri/oră). Ciclul zilnic este 1/24 = 0.042, cel săptămânal 1/168 = 0.006, iar armonica de jumătate de zi 1/12 = 0.083 cicluri/oră. Ca să izolați componenta zilnică, trece-banda trebuie să lase să treacă 0.042 și să le respingă pe celelalte două: `[1/30, 1/20]`.

```
sos_day = signal.butter(2, [1/30, 1/20], btype="bandpass", fs=1.0, output="sos")
daily = signal.sosfiltfilt(sos_day, vol.values)
```

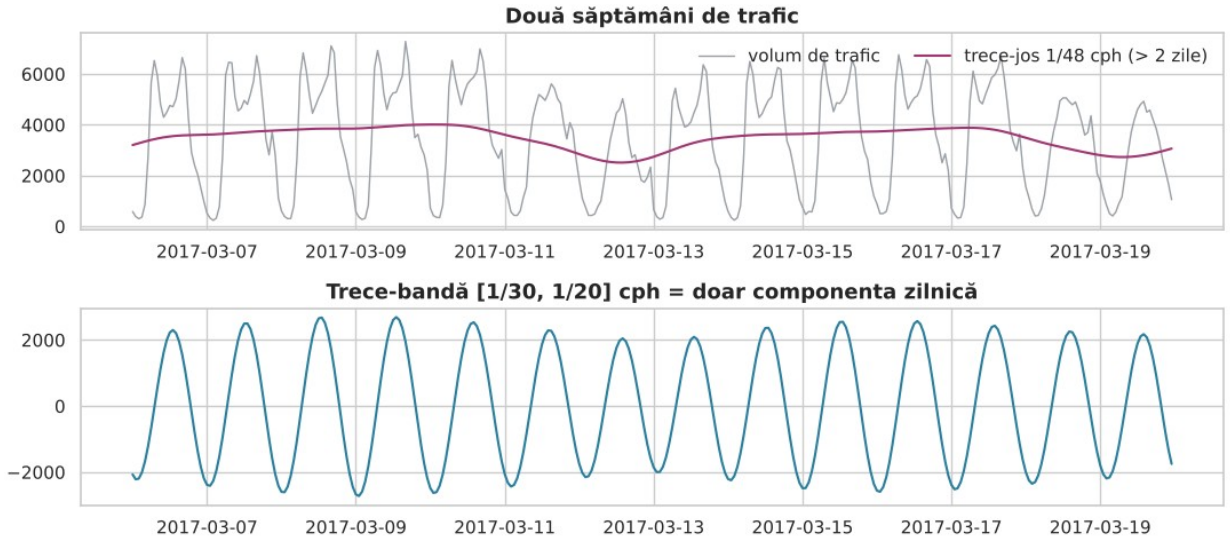


Figura 10. Două săptămâni de trafic: un trece-jos la 1/48 cicluri/oră păstrează scăderile din weekenduri; trece-banda returnează o sinusoidă zilnică curată (dubla creastă a orelor de vârf dispare odată cu tot ce nu se află la 24 h)

EEG și CO_2. Fișierul EEG conține deja puteri pe benzi o dată pe secundă, deci semnalul este lent și zgomotos: Savitzky–Golay și o medie mobilă de 21 s sunt candidatele. CO_2 din birou are defecțiuni izolate de un eșantion:

aici un median de 5 elimină vârful (2 400 → 732 ppm), în timp ce o medie mobilă de 5 îl reduce doar la 1 000 ppm și lasă o umflătură.

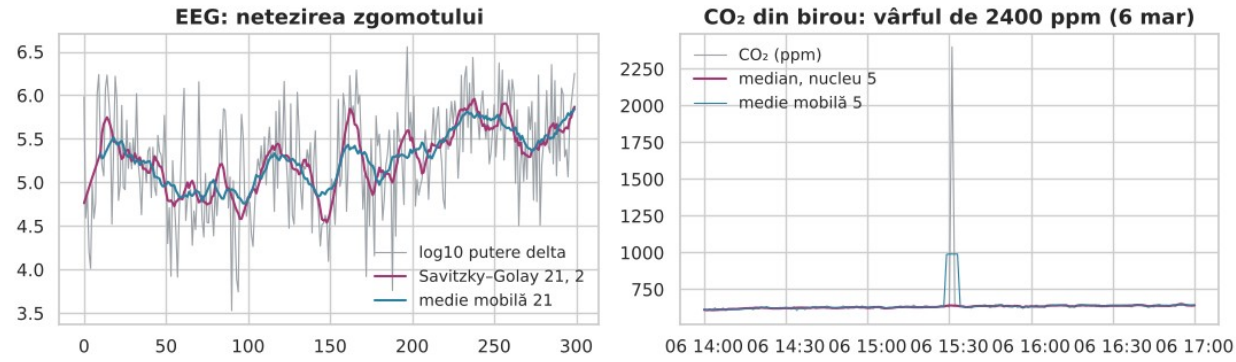


Figura 11. Stânga: puterea delta EEG netezită cu Savitzky–Golay și cu o medie mobilă. Dreapta: CO_2 din birou cu o defecțiune de 2 400 ppm

CE ÎNVĂȚĂM

(1) stabiliți banda în Hz; (2) alegeți tipul și frecvențele de tăiere; (3) luați cel mai mic ordin sau cea mai mică lungime care îndeplinește specificația; (4) verificați **răspunsul în frecvență**, nu doar curba filtrată; (5) decideți între cauzal și fază zero; (6) vârfurile izolate cer un median, nu un filtru trece-jos.

2 Proiectarea pasului de eliminare a tendinței

2.1 Ce fel de tendință și ce trebuie să rămână?

O **tendință** este o schimbare lentă a nivelului. Spectrele, corelațiile, detectoarele de puncte de schimbare, ARIMA și PCA pe ferestre presupun un nivel constant, așa că eliminăm mai întâi tendința. Trei întrebări decid metoda: *ce fel de tendință este* (o dreaptă, o curbă, o rătăcire de tip mers aleator, o linie de bază zilnică)? *Ce trebuie să rămână* (ciclul sezonier rămâne sau dispare, în funcție de scop)? *Cât de multă libertate dau modelului de tendință?* Fiecare metodă are **un singur număr** care controlează această libertate: gradul polinomului, fereastra, lag-ul diferențierii, frecvența de tăiere. Prea puțină libertate lasă tendința în date (subpotrivire); prea multă elimină chiar semnalul (supraantrenare).

Folosim o serie sintetică orară, de 14 zile, cu o tendință pătratică **cunoscută**, un ciclu de 24 de ore și zgomot, astfel încât fiecare alegere să poată fi evaluată față de adevăr.

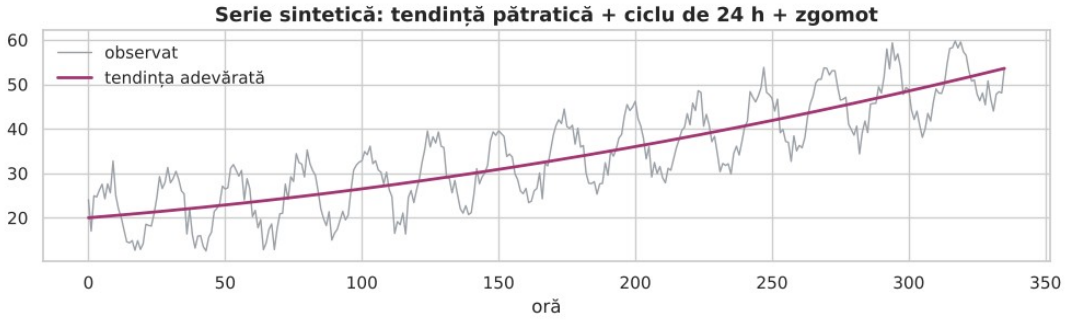


Figura 12. Seria sintetică: tendință pătratică + ciclu de 24 h + zgomot

2.2 Eliminarea tendinței polinomiale: gradul

`signal.detrend(y, type="linear")` scade o dreaptă; pentru orice alt grad folosiți `np.polynomial.Polynomial.fit(h, y, deg)` (rescalează intern `x`, ceea ce evită problemele numerice ale lui `np.polyfit` cu valori mari ale lui `x`). **Gradul se alege pe date reținute**: potriviți pe primele 80 % din serie și măsurați eroarea pe ultimele 20 %. Eroarea de antrenare scade cu fiecare grad în plus – întotdeauna – deci nu poate fi folosită pentru alegere.

```
p = np.polynomial.Polynomial.fit(h[:split], y[:split], deg)
eroare_test = np.sqrt(np.mean((y[split:] - p(h[split:])) ** 2))
```

REZULTAT

grad	0	1	2	3	4	5	6	7	8	9	10
rmse_antrenare	9.16	6.20	6.13	6.12	6.11	6.11	6.10	6.10	6.09	6.09	6.07
rmse_test	19.27	6.95	6.11	6.71	8.43	6.94	29.08	22.28	144.73	265.92	2701.51

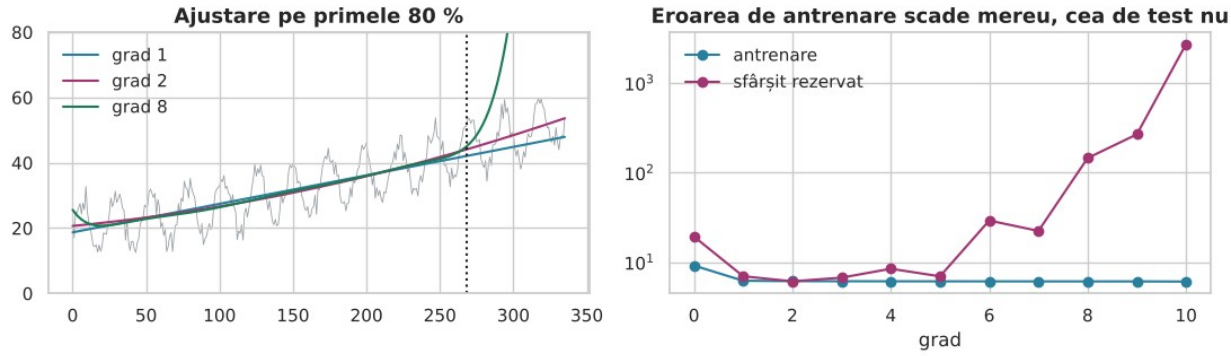


Figura 13. Stânga: gradul 8 se potrivește pe primele 80 % și explodează în afara lor. Dreapta: eroarea de antrenare continuă să scadă, cea pe date reținute nu

CE ÎNVĂȚĂM

Gradele 2, 3 și 5 au aproape aceeași eroare pe date reținute: alegeți-l pe **cel mai mic** (parcimonie). Un polinom de grad mare poate absorbi și părți lente ale sezonalității.

2.3 Media sau mediana mobilă: fereastra trebuie să fie multiplu al perioadei

O medie mobilă centrată cu fereastra `w` estimează nivelul local. Dacă `w` este **multiplu al perioadei sezoniere**, fiecare fereastră conține cicluri complete și ciclul se mediază la zero; orice altă valoare lasă o ondulație a componentei sezoniere în tendința estimată. O fereastră mai lungă mediază mai mult zgomot, dar nu poate urmări curbele tendinței și pierde `w/2` eșantioane la ambele capete.

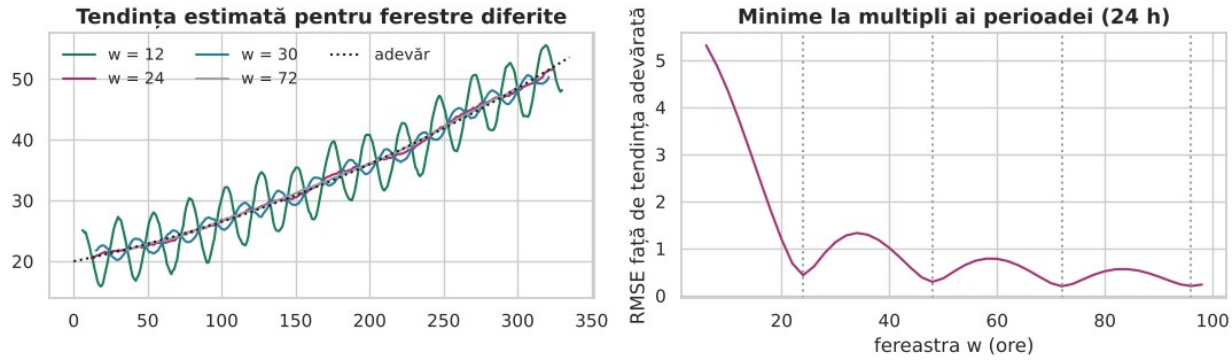


Figura 14. Stânga: tendința estimată cu ferestre diferite. Dreapta: eroarea față de tendința adevărată are minime la multiplii lui 24 h și este mare între ei

REZULTAT

	0	1	2	3
fereastră	12.00	24.00	30.00	72.0
RMSE	3.76	0.43	1.14	0.2

2.4 Diferențierea: lag, ordin și ce costă

`y.diff(lag)` calculează `y[t] − y[t − lag]`. Cu `lag = 1` elimină un nivel constant; de două ori, o tendință liniară; cu `lag = perioada` (diferențiere sezonieră) elimină un ciclu repetitiv și, odată cu el, o tendință lentă. Prețul este triplu: pierdeți `lag` eșantioane; **varianța zgomotului crește** (pentru zgomot alb o diferență o înmulțește cu 2, iar două diferențe cu 6); iar rezultatul este *variația* seriei, nu nivelul ei (o puteți anula cu `cumsum`).

REZULTAT

```
zgomot alb, raportul variantelor după 1 diferență : 1.96
zgomot alb, raportul variantelor după 2 diferențe: 5.8
```

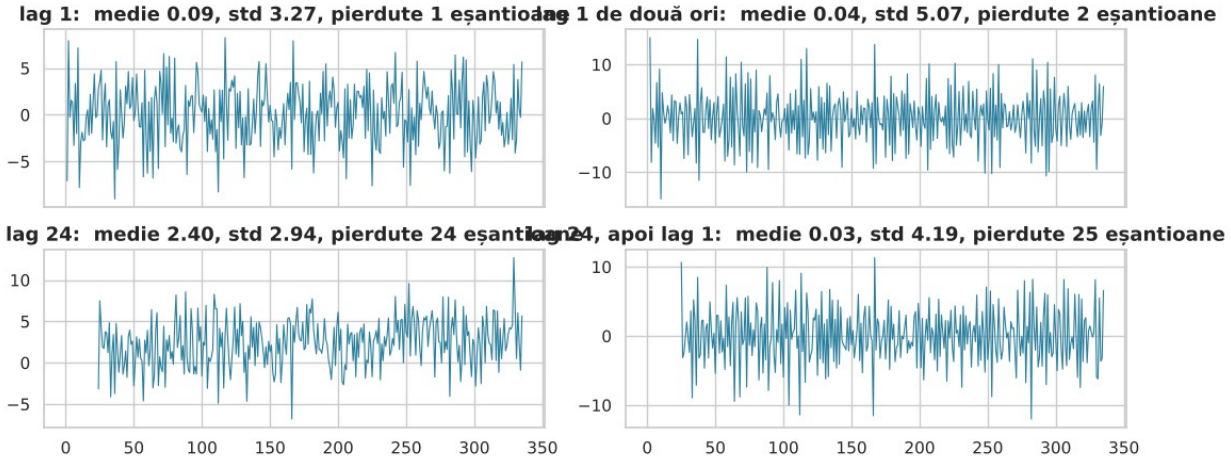


Figura 15. Patru scheme de diferențiere aplicate seriei: lag 24 elimină ciclul, lag 24 urmat de lag 1 face și media constantă

2.5 Trece-sus și eliminarea Fourier a tendinței: frecvența de tăiere

Tendința este „tot ce este mai lent decât cel mai lent fenomen care ne interesează”. Un **trece-sus** Butterworth cu frecvența de tăiere `fc` o elimină. Regula: puneți `fc` **cu mult sub cea mai joasă frecvență dorită** (ciclul de 24 h, 0.042 cicluri/oră) și deasupra tendinței, apoi verificați câștigul la frecvența dorită: trebuie să fie aproape de 0 dB. Eliminarea Fourier a tendinței anulează toți coeficienții FFT sub o frecvență-prag: este un trece-sus cu margine verticală, simplu, dar presupune un semnal periodic și creează artefacte la margini (vedeți capătul drept al liniei întrerupte).

REZULTAT

```
fc = 1/200 cph : câștig la ciclul zilnic =  -0.00 dB   (sosfiltfilt îl dublează:  -0.00
dB)
fc = 1/72 cph : câștig la ciclul zilnic =  -0.01 dB   (sosfiltfilt îl dublează:  -0.01 dB)
fc = 1/30 cph : câștig la ciclul zilnic =  -1.00 dB   (sosfiltfilt îl dublează:  -2.00 dB)
```

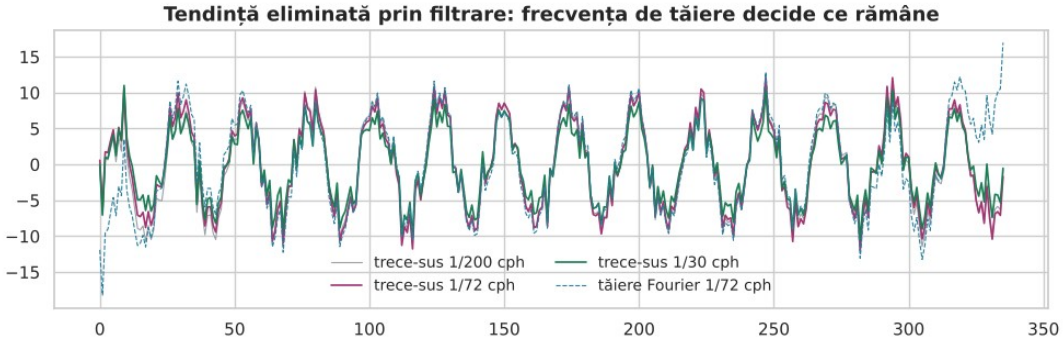


Figura 16. Eliminarea tendinței cu trece-sus pentru trei frecvențe de tăiere și cu Fourier: frecvența de tăiere decide ce rămâne

2.6 Staționară în jurul unei tendințe sau staționară prin diferențiere?

Două procese arată asemănător: o **tendință deterministă plus zgomot** (scoateți dreapta și seria este staționară) și un **mers aleator cu derivă** (tendința este zgomotul acumulat; scoaterea unei drepte lasă o curbă rătăcitoare,

doar diferențierea o face staționară). Testul **Dickey–Fuller augmentat** (`adfuller`) are ipoteza nulă *„există o rădăcină unitate (nestaționară)”*: o valoare p mică (< 0.05) înseamnă staționară.

REZULTAT

```
                        brut  după eliminarea unei drepte  după 1 diferență
tendință + zgomot     0.9631                       0.000               0.0
mers aleator + derivă 0.3060                       0.106               0.0
```

Pentru *tendință + zgomot*, scoaterea dreptei este suficientă (p ≈ 0); pentru *mersul aleator*, dreapta lasă p = 0.11 (nestaționar), în timp ce **doar o diferență** trece testul. Folosiți testul, nu imaginea, ca să alegeți.

2.7 STL: tendință, parte sezonieră și rest împreună

`STL` le estimează pe amândouă cu LOESS. Parametri: `period` (cunoscută), `seasonal` (impar, ≥ 7: cât de repede poate varia forma sezonieră; mai mare = mai rigid), `trend` (fereastră impară în eșantioane; mai mică urmărește schimbări mai rapide) și `robust=True` (reduce ponderea valorilor aberante). **Tăria** `1 − var(rest) / var(componentă + rest)` arată cât din serie explică o componentă.

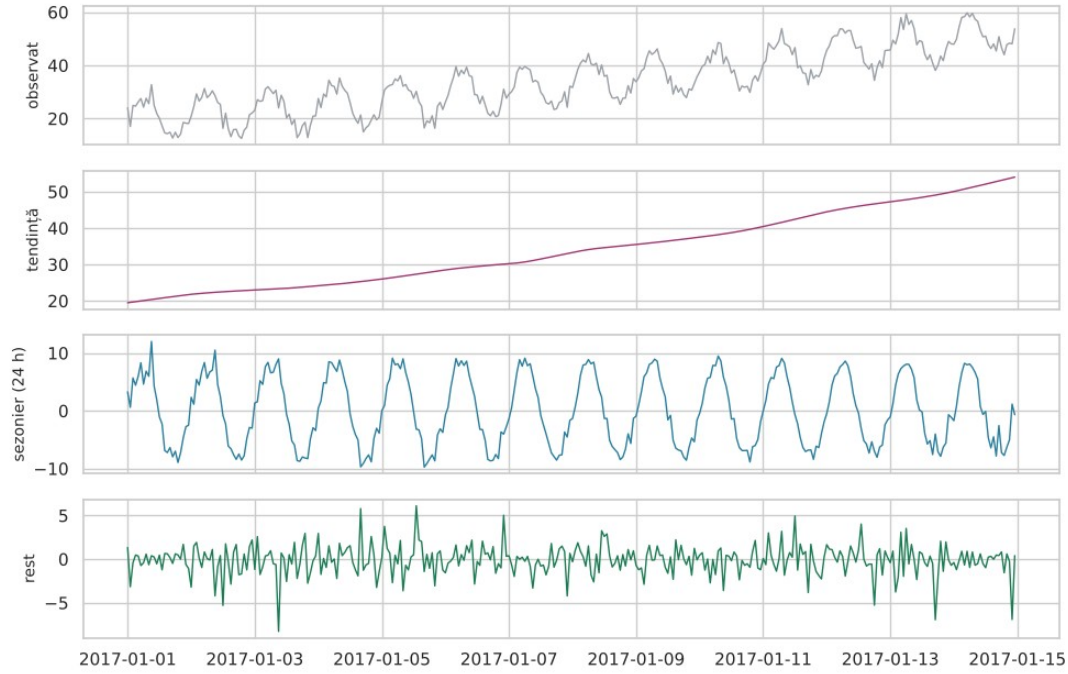


Figura 17. STL pe seria sintetică: tendința estimată are un RMSE de 0.39 față de cea adevărată

2.8 Date reale: trafic și EEG

Traficul are un tipar zilnic și unul săptămânal. O diferență sezonieră cu lag 24 lasă efectul de weekend; lag 168 le elimină pe amândouă: alegeți lag-ul cu abaterea standard rămasă mai mică (1 011 pentru lag 24, 574 pentru lag 168 în 2017). **EEG**: în timpul unei curse puterea de bază derivă (oboseală, contactul electrozilor). Vrem să păstrăm schimbările mai rapide de cinci minute, deci o mediană mobilă de 301 s este linia de bază, iar un trece-sus la 1/300 Hz face același lucru în domeniul frecvenței.

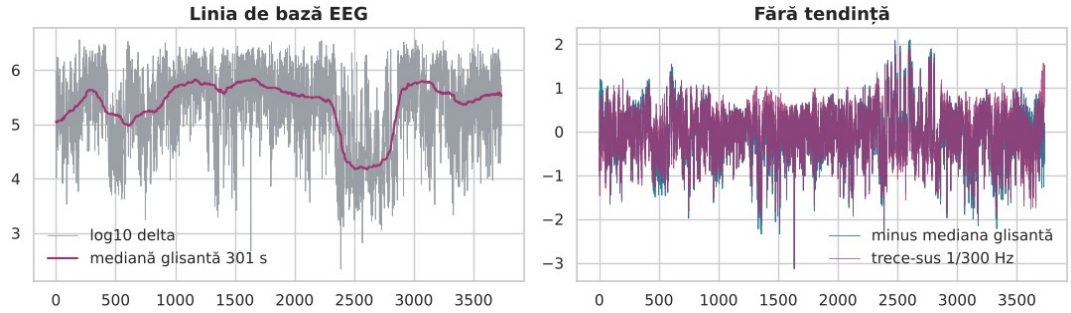


Figura 18. Puterea delta EEG (log10): linia de bază dată de mediana mobilă de 301 s și seria fără tendință (versiunile cu mediană și cu trece-sus coincid)

CE ÎNVĂȚĂM

Gradul / fereastra / lag-ul / frecvența de tăiere este un **buton de compromis părtinire–variață**: alegeți-l dintr-un criteriu de validare (eroare pe date reținute, ondulația la perioadă, testul ADF, câștigul la frecvența dorită), niciodată din „arată plat”. O fereastră mai scurtă dă întotdeauna un rezidual mai mic – tocmai de aceea reziduul nu este un criteriu.

3 Proiectarea interpolării valorilor lipsă

3.1 Mai întâi priviți golurile

Valorile lipsă sunt de trei feluri și cer tratamente diferite: **eșantioane izolate** (un pachet pierdut; aproape orice metodă merge), **goluri** (un senzor oprit minute sau ore; metoda contează, iar lungimea decide dacă umplerea are sens) și valori lipsă **la început sau la sfârșit** (extrapolare, de obicei nepermisă). Fișierul de trafic pare complet – 48 204 rânduri – dar conține 7 629 de marcaje de timp duplicate și, după reindexarea pe o grilă orară regulată, 47 de ore lipsă în 2017.

```
grid = pd.date_range("2017-01-01", "2017-12-31 23:00", freq="h")
vol = tr.loc["2017", "traffic_volume"].reindex(grid)
isna = vol.isna(); run_id = (isna != isna.shift()).cumsum()
goluri = isna.groupby(run_id).agg(["first", "size"]).query("first")["size"]
```

REZULTAT

```
rânduri: 48204   momente de timp duplicate: 7629
ore lipsă: 47   număr de goluri: 21   cel mai lung gol: 9 h
size
1    15
3     2
4     1
6     1
7     1
9     1
```

3.2 Metodele și parametrii lor

Metodă	Apel	Parametru	Comportament
ultima observație reportată	`s.ffill(limit=)`	`limit`	potrivită pentru *stări*, greșită pentru semnale netede
cel mai apropiat vecin	`interpolate("nearest")`	—	trepte
liniară	`interpolate("linear")` sau `"time"`	`limit`, `limit_area`	alegere implicită robustă; taie colțurile
spline cubic	`CubicSpline(x, y)`	condiție la limită	neted, dar **depășește** valorile
PCHIP	`PchipInterpolator`	—	neted, nu iese niciodată din intervalul datelor
Akima	`Akima1DInterpolator`	—	neted, oscilații mici
copie sezonieră	`s.fillna(s.shift(period))`	`period`	folosește aceeași oră de ieri / din săptămâna trecută

`limit` este un **parametru de siguranță**: „nu umple mai mult de `limit` eșantioane consecutive”. `limit_area="inside"` umple doar golurile dintre două valori valide (fără extrapolare). Clasa veche `interp1d` este depășită în SciPy: folosiți clasele de mai sus sau `np.interp`. Testăm metodele pe un semnal sintetic cu adevăr cunoscut (o sinusoidă lentă cu perioada 60 și una mai rapidă cu perioada 17), tăind goluri de 2, 8 și 45 de eșantioane.

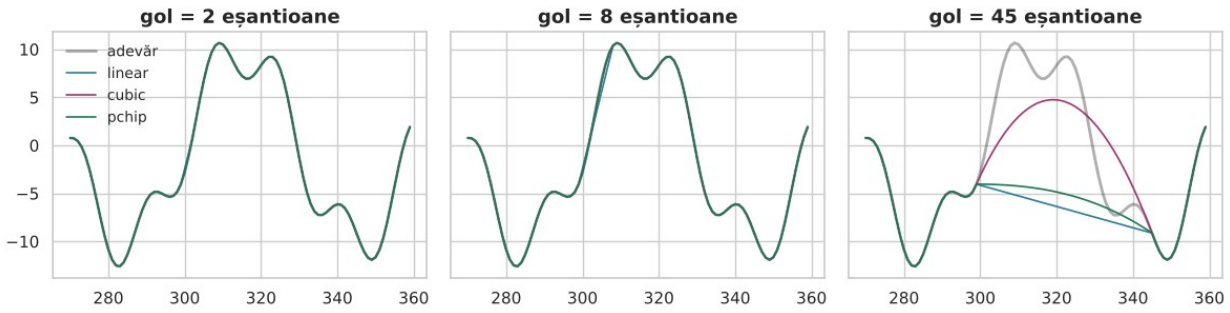


Figura 19. Goluri de 2, 8 și 45 de eșantioane: pentru golul lung toate metodele eșuează – informația nu se află în date

REZULTAT

```
gap        2     8      45
method
akima    0.04  0.30   5.90
cubic    0.01  0.09   4.64
ffill    2.19  8.78   9.32
linear   0.34  0.78  10.72
nearest  1.60  3.98  10.42
pchip    0.07  0.04   9.73
```

CE ÎNVĂȚĂM

Gol scurt: orice metodă netedă este bună. Gol mediu: metodele de tip spline câștigă dacă golul este mult mai scurt decât perioada celei mai rapide componente. Gol lung (trei sferturi din perioada lentă): **toate** eșuează. Regulă practică: aveți încredere în interpolare când golul este mai scurt de aproximativ **un sfert din perioada celei mai rapide componente care vă interesează**; pentru goluri mai lungi folosiți un model (copie sezonieră, alt senzor) și **marcați** valorile ca imputate.

3.3 Depășirea și marcajele de timp neregulate

Un spline cubic este neted, dar poate ieși din intervalul datelor lângă un salt – periculos pentru mărimi care nu pot fi negative (numărători, umiditate, ocupare). PCHIP și interpolarea liniară nu fac asta niciodată. Când marcajele de timp **nu sunt echidistante**, interpolarea după index este greșită: folosiți `method="time"` (indexul trebuie să fie `DatetimeIndex`) sau `np.interp(x_nou, x_cunoscut, y_cunoscut)`.

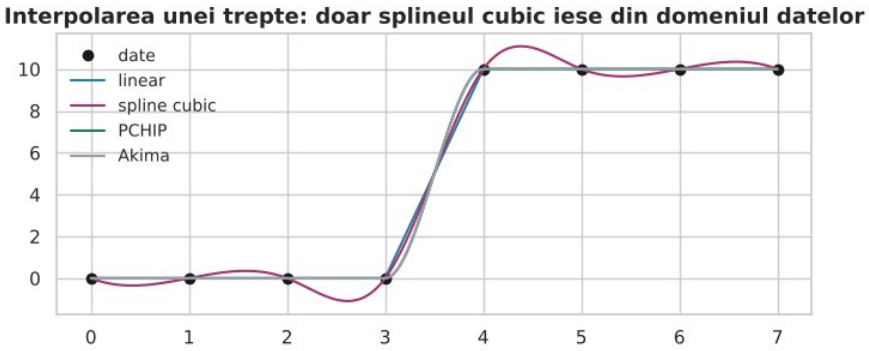


Figura 20. Interpolarea unei trepte: doar splineul cubic iese din intervalul datelor (maximum 11.09, minimum −1.09)

REZULTAT

```
                     brut  liniar (după poziție)  time (după moment de timp)
2026-01-01 00:00:00  10.0                   10.0                   10.000000
2026-01-01 00:10:00   NaN                   15.0                   11.666667
2026-01-01 01:00:00  20.0                   20.0                   20.000000
2026-01-01 01:10:00   NaN                   30.0                   30.000000
2026-01-01 01:20:00  40.0                   40.0                   40.000000
```

Valoarea de la 00:10 este 10 + (20 − 10)·(10/60) = 11.67 după timp, dar 15 după poziție, care ignoră faptul că între 00:10 și 01:00 trec 50 de minute.

3.4 Evaluarea cinstită: mascați valori cunoscute, umpleți, măsurați

Pe date reale nu există adevăr, așa că îl **creăm**: ascundem 10 % aleatoriu din valorile *cunoscute* (izolate) și un bloc contiguu de 6 ore, le umplem cu mai multe metode și calculăm eroarea doar pe valorile ascunse. Experimentul trebuie să imite golurile reale (fracțiune, lungimea blocului).

REZULTAT

```
                            10 % izolate  bloc de 6 ore (14 iun, 08–13 h)
linear                             443.0                           458.0
pchip                              361.0                           391.0
aceeași oră ieri                  1031.0                           148.0
aceeași oră săptămâna trecută      594.0                           211.0
ffill                              907.0                           638.0
```

CE ÎNVĂȚĂM

Pentru ore izolate lipsă orice metodă netedă merge (PCHIP este cea mai bună). Pentru un **bloc de 6 ore în ora de vârf de dimineață**, interpolarea liniară taie vârful (RMSE 458); copierea *aceleiași ore din ziua precedentă* păstrează forma (RMSE 148). Metoda potrivită depinde de lungimea golului în raport cu structura sezonieră.

3.5 Imputare multivariată: KNN – și o greșeală clasică

`KNNImputer(n_neighbors=k)` umple o valoare lipsă cu media celor `k` rânduri cele mai apropiate, unde „apropiat” se măsoară pe **celelalte coloane**. Două reguli: dați-i mai multe coloane informative (cu o singură coloană returnează media coloanei – o linie plată!) și **scalați** coloanele, deoarece distanța amestecă unități. Pe datele biroului inteligent ascundem 31 de minute de temperatură într-o zi în care cunoaștem adevărul:

REZULTAT

```
lipsă: {'temp_c': 31, 'humidity_pct': 31, 'co2_ppm': 31, 'occupancy': 0, 'temp_out_c': 0}
întrerupere de la 2026-03-05 10:20:00 până la 2026-03-05 10:50:00 (31 min)
liniar                   0.107
KNN, 1 coloană           2.677
KNN, 4 coloane scalate   0.132
aceeași oră ieri         0.198
```

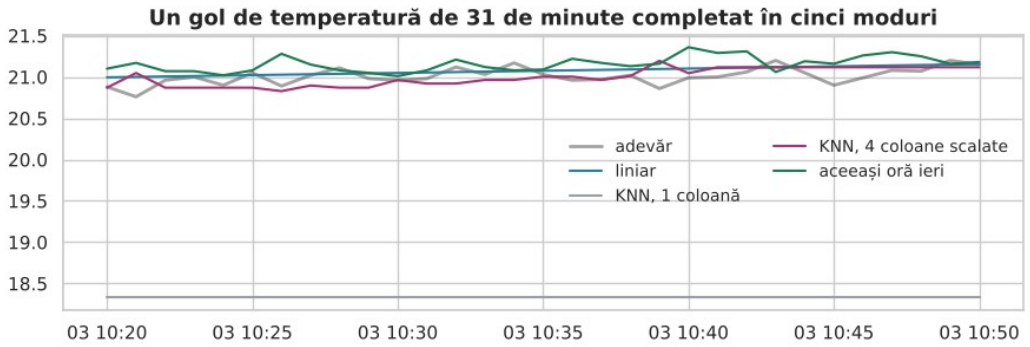


Figura 21. Un gol de temperatură de 31 de minute umplut în cinci feluri: KNN pe o singură coloană este o linie plată la media coloanei

CE ÎNVĂȚĂM

Pentru un gol scurt într-un senzor neted, **interpolarea liniară este adesea la fel de bună ca KNN și mult mai simplă**. Folosiți KNN când mai mulți senzori corelați lipsesc în intervale diferite. (1) priviți lungimile golurilor; (2) goluri scurte: liniar sau PCHIP, goluri lungi: model și marcaj; (3) protejați-vă cu `limit`; (4) validați prin mascare și raportați un RMSE pe metodă și pe lungime de gol.

4 Proiectarea unui detector de valori aberante

4.1 Defecțiuni, extreme și rolul cunoștințelor de domeniu

O **valoare aberantă** este o valoare care nu aparține procesului care le-a produs pe celelalte. Două cauze cer două tratamente: o **defecțiune** (un vârf de senzor, −5 °C în interior, 9 831 mm de ploaie într-o oră) trebuie eliminată sau reparată; un **extrem real** (un ambuteiaj, o explozie de activitate musculară în EEG) poate fi exact ceea ce căutăm. Un detector poate spune doar „neobișnuit”: *decizia* este a voastră și pornește de la **cunoștințele de domeniu** – limite fizice și domenii ale senzorilor – înainte de orice statistică. Fiecare detector are un **prag**; întrebarea de proiectare este mereu *câte alarme false și câte omisiuni pot accepta?* O măsurăm cu precizie, recall și F1 pe date sintetice în care am plantat noi valorile aberante.

4.2 Reguli globale: z-score, MAD și IQR

Regulă	Marchează dacă	Parametru tipic	Slăbiciune
z-score	`|x − medie| / std > k`	`k` = 3	media și abaterea standard sunt umflate de valorile aberante (mascare)
z-score modificat	`0.6745·|x − mediană| / MAD > 3.5`	3.5	cere un MAD nenul
IQR (Tukey)	`x < Q1 − c·IQR` sau `x > Q3 + c·IQR`	`c` = 1.5 (moderat), 3 (departe)	presupune o distribuție aproximativ simetrică

REZULTAT

```
        date   regula  marcate  precizie  recall   F1
 1% aberante   z, k=2       18      0.56    1.00 0.71
 1% aberante   z, k=3       10      1.00    1.00 1.00
 1% aberante   z, k=4       10      1.00    1.00 1.00
 1% aberante MAD, 3.5       12      0.83    1.00 0.91
 1% aberante IQR, 1.5       18      0.56    1.00 0.71
 1% aberante   IQR, 3       10      1.00    1.00 1.00
15% aberante   z, k=2      126      1.00    0.84 0.91
15% aberante   z, k=3       16      1.00    0.11 0.19
15% aberante   z, k=4        0      0.00    0.00 0.00
15% aberante MAD, 3.5      150      1.00    1.00 1.00
15% aberante IQR, 1.5      152      0.99    1.00 0.99
15% aberante   IQR, 3      150      1.00    1.00 1.00
```

CE ÎNVĂȚĂM

Cu 1 % valori aberante orice regulă funcționează, dar `k = 2` marchează aproximativ 5 % dintr-un eșantion normal: precizia lui se prăbușește. Cu **15 %** valori aberante abaterea standard este atât de umflată încât z-score *ratează* 89 % dintre ele (recall 0.11), în timp ce MAD și IQR rămân fiabile. Dacă nu cunoașteți contaminarea, folosiți **regulile robuste**.

Date asimetrice. Regulile IQR și z-score presupun simetrie. Numărătorile, duratele, ploaia și puterile pe benzi EEG sunt asimetrice la dreapta, deci coada lor dreaptă normală este marcată. Transformați întâi (`log`), detectați, apoi interpretați.

REZULTAT

```
date log-normale: IQR(1.5) marchează 99 valori; pe log(date): 14 valori
```

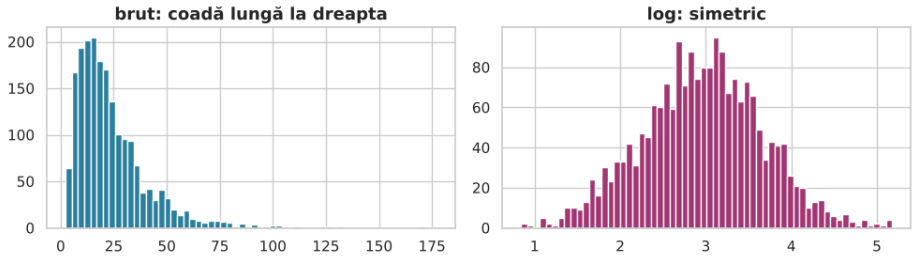


Figura 22. Date log-normale: valorile brute au o coadă lungă la dreapta (99 de valori marcate de regula IQR); logaritmul este simetric (14)

4.3 Valori aberante într-o serie de timp: filtrul Hampel

Un prag global eșuează când **nivelul se schimbă**: o valoare normală de după-amiază poate fi „neobișnuită” pentru întreaga serie. **Filtrul Hampel** aplică regula MAD într-o fereastră glisantă: un eșantion este aberant dacă `| x − mediana mobilă| > n_sigmas · 1.4826 · MAD mobil`. Fereastra trebuie să fie **suficient de lungă încât să conțină mai multe eșantioane normale de fiecare parte a unui vârf** (cel puțin 2 × cel mai lung vârf + 1) și **suficient de scurtă încât nivelul să fie aproape constant în interiorul ei**. Un **prag minim de scară** (cea mai mică scară acceptată, de exemplu zgomotul senzorului) evită alarmele false pe porțiunile plate unde MAD este zero.

```
def hampel(s, window=11, n_sigmas=3, floor=0.0):
    s = pd.Series(s); med = s.rolling(window, center=True, min_periods=1).median()
    mad = 1.4826 * (s - med).abs().rolling(window, center=True,
min_periods=1).median()
    return ((s - med).abs() > n_sigmas * np.maximum(mad, floor)).values
```

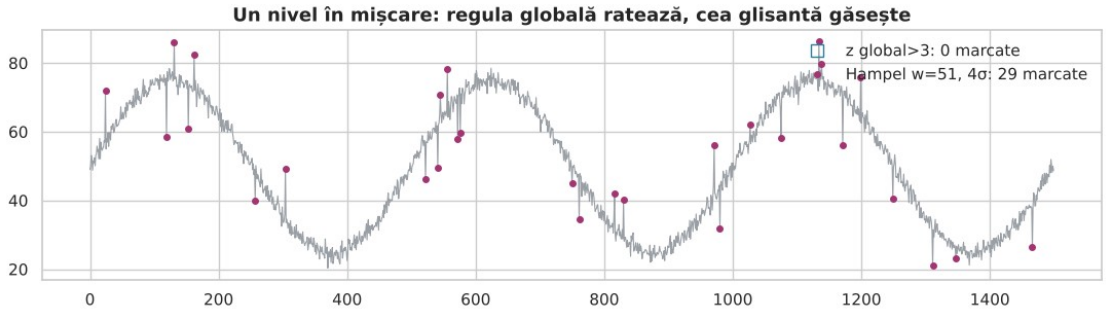


Figura 23. O serie cu nivel mobil și 25 de valori aberante plantate: z-score global nu găsește niciuna, filtrul Hampel le găsește

REZULTAT

```
n_sigmas      2     3     4
fereastră
5          0.20  0.29  0.31
11         0.28  0.49  0.67
21         0.28  0.60  0.86
51         0.35  0.79  0.93
201        0.38  0.79  0.75
z global>3 : [0.0, 0.0, 0.0]
```

Grila F1 arată ambele moduri de eșec: o **fereastră prea mică** (5) urmărește chiar vârful, o **fereastră prea mare** (201) nu mai vede nivelul local; `n_sigmas = 2` dă alarme false, `4` este sigur când vârfurile sunt mari.

4.4 Detectoare multivariate: Isolation Forest, LOF, DBSCAN

Un rând poate fi normal în fiecare coloană și neobișnuit în *combinație* (un CO_2 ridicat cu ocupare zero). **Isolation Forest** are parametrul `contamination`: *fracțiunea de rânduri de marcat*. Ea **nu este estimată** de model: spuneți 0.15 și marchează 15 % indiferent de date. **Local Outlier Factor** are `n_neighbors` (mic = anomalii locale, dar zgomotos). **DBSCAN** are `eps` (raza) și `min_samples`; punctele care nu se află în nicio regiune densă sunt etichetate zgomot (−1). Cere trăsături **scalate**; alegeți `eps` la cotul distanțelor sortate până la al k-lea vecin.

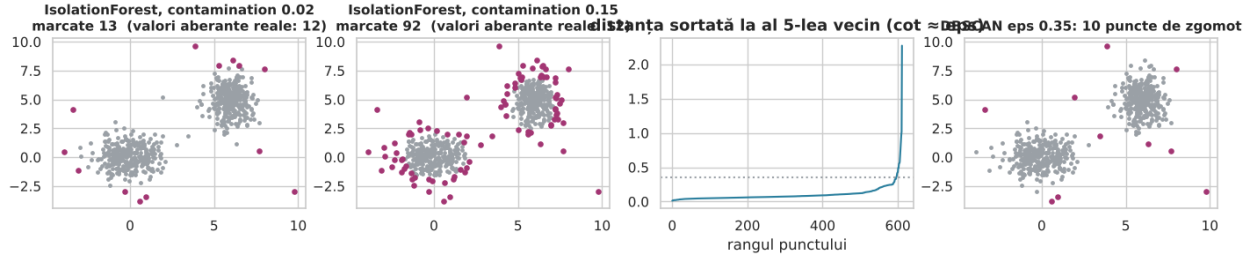


Figura 24. Doi Isolation Forest (contamination 0.02 și 0.15), curba distanțelor la al k-lea vecin și DBSCAN cu eps = 0.35 pe 12 valori aberante plantate

REZULTAT

```
eps  0.1: zgomot 119  clustere 6  precizie 0.09  recall 0.92
eps  0.2: zgomot  37  clustere 2  precizie 0.22  recall 0.67
eps 0.35: zgomot  10  clustere 2  precizie 0.70  recall 0.58
eps  0.6: zgomot   4  clustere 1  precizie 1.00  recall 0.33
eps  1.0: zgomot   1  clustere 1  precizie 1.00  recall 0.08
```

4.5 Ce facem după detecție

Acțiune	Când
se pun pe `NaN` și se interpolează (Capitolul 3)	defecțiuni de senzor în semnale netede
limitare (`clip(lower, upper)`, winsorizare)	valori reale, dar extreme, care ar domina un model
se păstrează și se marchează (o coloană booleană nouă)	veți modela anomaliile sau nu sunteți sigur
se șterge rândul	doar pentru observații independente, niciodată într-o serie de timp regulată

4.6 Date reale: biroul inteligent și traficul

Mai întâi regulile de domeniu. Un CO_2 sub 300 ppm (nivelul exterior este în jur de 420) sau peste 1 500 ppm cu cel mult 12 persoane, și o temperatură interioară de −5 °C, sunt fizic neplauzibile. *Apoi statistica:* Hampel cu fereastră de 15 minute (vârfurile durează 1–2 eșantioane) și 5σ. Pragul minim contează: cu 3 ppm detectorul marchează 14 mici oscilații inofensive în porțiunile liniștite; cu 8 ppm – de câteva ori zgomotul senzorului – marchează exact cele șase defecțiuni.

REZULTAT

```
regula fizică        : 6 valori -> [150.0, 150.0, 2100.0, 2250.0, 2250.0, 2400.0]
Hampel 15, 5σ, prag minim 3: 20 valori (14 alarme false în porțiunile liniștite: MAD este
acolo ~…
Hampel 15, 5σ, prag minim 8: 6 valori -> [150.0, 150.0, 2100.0, 2250.0, 2250.0, 2400.0]
z global>3           : 6 valori
temperatură <0 : 6 eșantioane la 2026-03-07 06:40:00

trafic: temp == 0 K -> 10 rânduri;  rain_1h max = 9831.3 mm (următorul ca mărime este
55.63 mm)
traffic_volume == 0 -> 2 rânduri
```

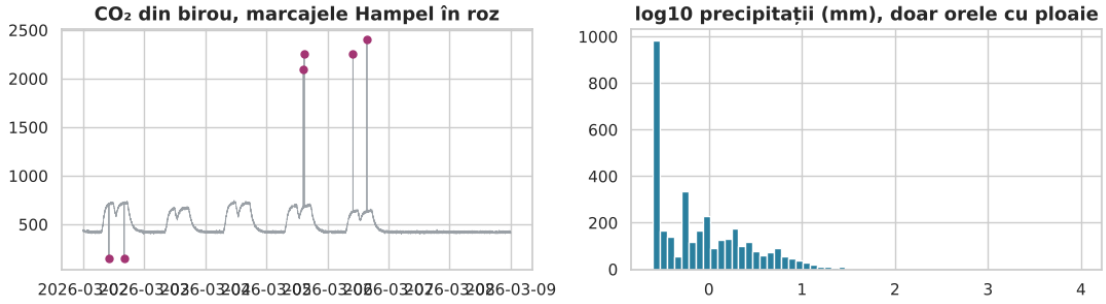


Figura 25. Stânga: CO_2 din birou cu detecțiile Hampel în roz. Dreapta: ploaia din fișierul de trafic (scară logaritmică, doar orele cu ploaie); o valoare este 9 831 mm

CE ÎNVĂȚĂM

Începeți cu fizica, apoi o regulă **robustă** (MAD / Hampel), reglați pragul pe valori aberante **plantate** și tratați *fracțiunea marcată* ca pe un număr de inspectat: un 15 % neașteptat este o greșeală a parametrului, nu a datelor.

5 Proiectarea reducerii dimensionalității

5.1 Extragere sau selecție?

Prea multe trăsături dăunează: modelele se supraantrenează, distanțele devin lipsite de sens, graficele sunt imposibile. Există două familii de remedii. **Extragerea de trăsături** construiește trăsături *noi* din combinații ale celor vechi (PCA, autoencoder, t-SNE); **selecția de trăsături** *păstrează un subset* dintre cele vechi (variață, corelație, teste univariate, penalizare L1). Întrebările de proiectare sunt: *câte dimensiuni păstrez? În ce scară sunt măsurate trăsăturile? Este structura liniară? Va folosi un model rezultatul* – caz în care reducerea trebuie potrivită în interiorul validării încrucișate, doar pe partea de antrenare.

Setul nostru sintetic are **trei factori ascunși** care generează zece trăsături observate, măsurate în unități foarte diferite (abateri standard de la 1 la 1 000). Răspunsul corect este deci cunoscut: trei componente.

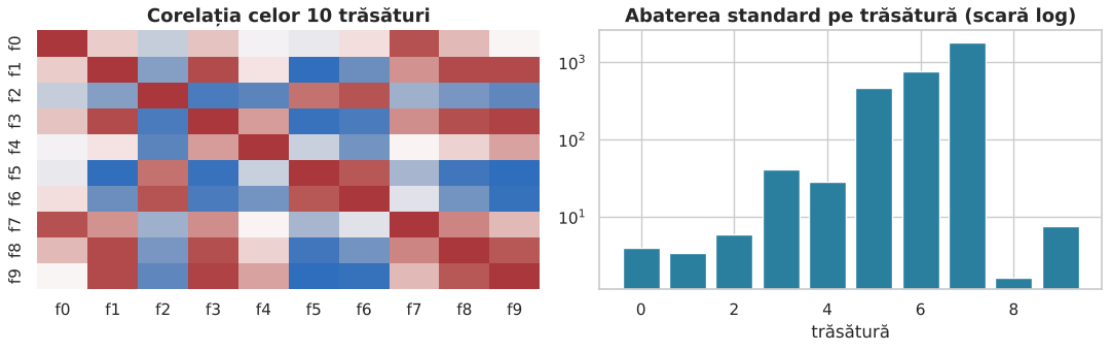


Figura 26. Corelația celor zece trăsături și abaterile lor standard pe scară logaritmică

5.2 PCA: întâi scalați, apoi alegeți numărul de componente

PCA găsește direcțiile de **variață maximă**, iar variața depinde de **unități**: o trăsătură măsurată în mii domină prima componentă doar din cauza unității. **Standardizați mai întâi** (`StandardScaler`), cu excepția cazului în care toate trăsăturile au aceeași unitate. Câte componente `k`? Patru reguli, care ar trebui să fie de acord: **varianța cumulată** atinge 90 % (`PCA(n_components=0.90)`); cotul din **graficul scree**; **Kaiser** (valoare proprie > 1 pe date standardizate); **analiza paralelă** (valoarea proprie depășește pe cea a zgomotului pur de aceeași formă).

REZULTAT

```
ponderea PC1  date brute: 0.82   standardizat: 0.66
trăsătura cu cel mai mare |loading| pe PC1 (brut): 7   (trăsătura cu cea mai mare unitate)
componente cu varianță cumulată >= 90 %: 3
Kaiser (valoare proprie > 1): 3    analiză paralelă: 3
```

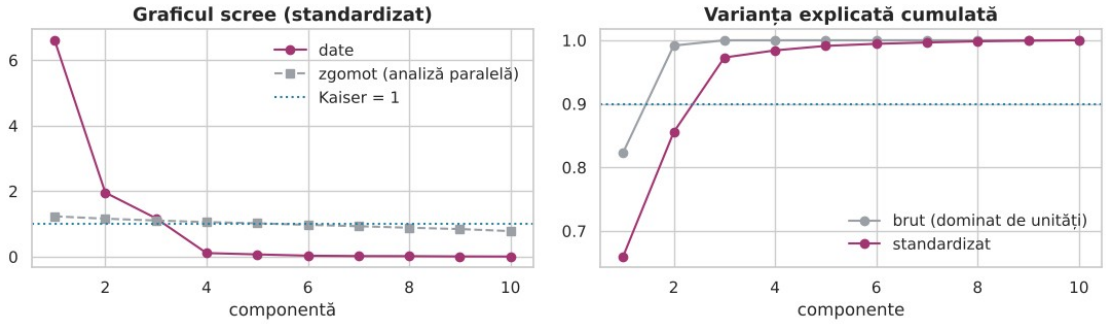


Figura 27. Stânga: graficul scree față de zgomot (analiza paralelă) și linia Kaiser. Dreapta: varianța cumulată, date brute (dominate de o unitate) și standardizate

REZULTAT

```
k =  1: RMSE de reconstrucție = 0.583
k =  2: RMSE de reconstrucție = 0.380
k =  3: RMSE de reconstrucție = 0.165
k =  5: RMSE de reconstrucție = 0.094
k = 10: RMSE de reconstrucție = 0.000
```

Eroarea de reconstrucție arată ce pierdeți: scade de 3.5 ori de la `k = 1` la `k = 3` și apoi se aplatizează. O reducere sub dimensiunea „adevărată” distruge structura; peste ea păstrează doar zgomot.

5.3 Selecția de trăsături: fiecare prag este un parametru

Regulă	Parametru	Atenție
`VarianceThreshold(t)`	varianța minimă `t`	folosiți-o pentru a găsi coloanele **constante sau aproape constante** *înainte* de scalare: după standardizare toate coloanele au varianța 1 și regula este oarbă
filtru de corelație	pragul `|r|` (0.9)	păstrați o coloană din fiecare pereche corelată, pe cea mai legată de țintă
`SelectKBest(f_classif, k)`	`k`	scoruri calculate doar pe fold-ul de antrenare
penalizare L1 (`l1_ratio=1`)	`C` (mai mic = mai rar)	cere trăsături scalate

REZULTAT

```
varianța celor două coloane inutile: {'const': 0.0, 'rare_flag': 0.0083}
VarianceThreshold(0.01) pe coloane brute  păstrează 10 din 12  (le elimină pe ambele)
VarianceThreshold(0.01) după StandardScaler păstrează 11 din 12  (indicatorul rar are acum
varian…
k                 1      2     3      4      5      6     7     8      9     10  PCA 1
PCA 2  PC…
acuratețe cv  0.908  0.938  0.94  0.953  0.952  0.957  0.95  0.95  0.955  0.957   0.82
0.928  0.…
```

SCURGEREA DE INFORMAȚIE

Ca să alegeți `k` sau `C`, **validați încrucișat întregul pipeline** (scaler → selector → model). Dacă selectați trăsăturile pe toate datele și abia apoi validați încrucișat, selecția a văzut deja fold-urile de test, iar rezultatul este optimist. Pe 100 de rânduri cu 500 de trăsături aleatorii și etichete aleatorii, selectarea a zece trăsături în afara validării încrucișate dă o acuratețe de aproximativ 80 % – pentru etichete care nu poartă nicio informație (Exercițiul 5.5).

5.4 Reducere neliniară: t-SNE și autoencoderul

t-SNE este doar pentru vizualizare: `perplexity` este aproximativ numărul efectiv de vecini (5–50, mereu sub n/3). Prea mic dă multe insule minuscule, prea mare o pată; distanțele dintre clustere și mărimea lor *nu* au semnificație, iar un t-SNE potrivit nu poate fi aplicat pe date noi. Un **autoencoder** este o rețea cu un gât de `k` unități antrenată să-și reproducă intrarea. Cu activări liniare învață același subspațiu ca PCA; cu o activare neliniară poate urmări structuri **curbe**. `k` se alege după eroarea de reconstrucție, exact ca la PCA.

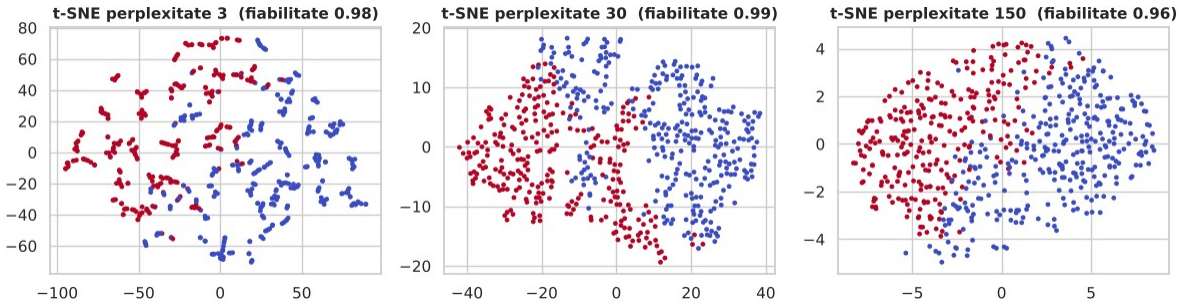


Figura 28. t-SNE cu perplexity 3, 30 și 150 și încrederea (trustworthiness) vecinătăților

REZULTAT

```
autoencoder 3 → 1 → 3, activare identity: RMSE de reconstrucție = 0.492
autoencoder 3 → 1 → 3, activare tanh    : RMSE de reconstrucție = 0.032
PCA cu 1 componentă                : RMSE de reconstrucție = 0.492
```

Pentru o structură unidimensională curbă, PCA poate folosi doar o dreaptă – la fel și un autoencoder *liniar* (eroare 0.49) – în timp ce un gât cu unități `tanh` urmărește curba (eroare 0.03). Pe datele sintetice liniare de mai sus modelul neliniar nu ar aduce niciun avantaj.

5.5 Date reale: puterile pe benzi EEG

Opt puteri pe benzi sunt asimetrice la dreapta pe mai multe ordine de mărime: câteva eșantioane cu putere uriașă domină varianța. Pipeline-ul de reducere este deci **log10 → standardizare → PCA**. Evaluarea cinstită a unui clasificator pe o serie de timp cere fold-uri **contigue**: fold-urile amestecate pun secunde vecine – aproape același eșantion – atât în antrenare, cât și în test.

REZULTAT

```
ponderea PC1  brut+standardizat: 0.47    log10+standardizat: 0.52
componente pentru 90 % varianță  brut: 5    log10: 5
            PC1   PC2   PC3
delta      0.27 -0.40  0.71
theta      0.40 -0.33  0.17
lowAlpha   0.40  0.31  0.14
highAlpha  0.36 -0.04 -0.19
lowBeta    0.36  0.52  0.22
highBeta   0.40 -0.10 -0.37
lowGamma   0.34  0.34 -0.21
highGamma  0.26 -0.49 -0.42
acuratețea clasei majoritare    : 0.57
fold-uri contigue (oneste)     : {1: 0.49, 2: 0.45, 4: 0.42, 8: 0.46}
fold-uri amestecate (cu scurgere, prea bune): {1: 0.61, 2: 0.65, 4: 0.66, 8: 0.69}
```

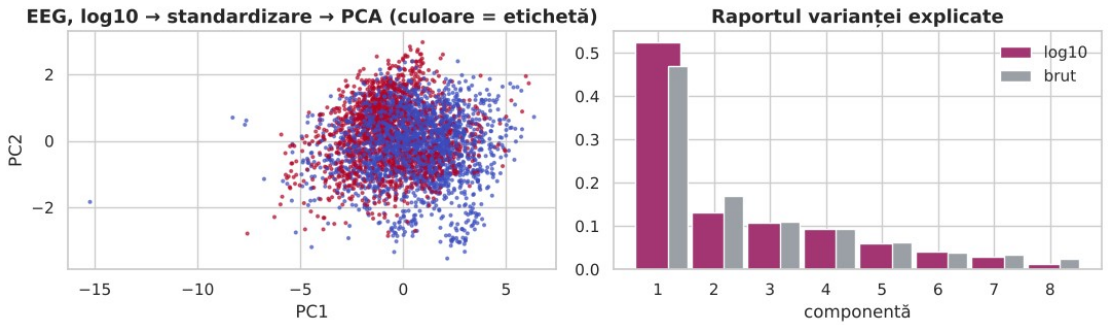


Figura 29. Stânga: EEG după log10, standardizare și PCA (culoare = etichetă). Dreapta: varianța explicată cu și fără logaritm

CE ÎNVĂȚĂM

(1) scalați – și aplicați logaritm datelor asimetrice – înainte de PCA; (2) alegeți k după mai multe reguli și după eroarea de reconstrucție; (3) potriviți scalere, selectoare și PCA **în interiorul** validării încrucișate; (4) folosiți t-SNE doar ca să priviți; (5) pe acest EEG acuratețea cinstită este *sub* linia de bază a clasei majoritare (0.57), în timp ce fold-urile amestecate raportează 0.61–0.69: puterile pe benzi ale unei curse nu se generalizează între segmentele etichetate – o constatare importantă despre date, nu un eșec al PCA.

6 Proiectarea unui detector de puncte de schimbare

6.1 Ce se schimbă și de ce am nevoie?

Un **punct de schimbare** este un moment în care comportamentul statistic al unei serii se schimbă: media, varianța, panta sau conținutul în frecvență (șoferul adoarme, o mașină pornește, o clădire își schimbă regimul). Un detector răspunde la trei întrebări – *există o schimbare? unde? câte?* – iar fiecare metodă echilibrează alarmele false cu întârzierea printr-**un singur parametru**. Lista de proiectare:

- **Ce se schimbă?** Media → cost `l2`; media și varianța → `normal`; orice → `rbf`; panta → cost liniar pe porțiuni sau diferența seriei.
- **Offline sau online?** Offline (toată seria este disponibilă): PELT, segmentare binară. Online (alarmă cât mai repede): CUSUM.
- **Câte schimbări?** Cunoscut → `n_bkps`; necunoscut → o **penalitate**.
- **Cât de aproape de adevăr trebuie să fie locația?** → *marja* evaluării și `min_size`.
- **Ce trebuie curățat mai întâi?** Valorile aberante creează schimbări false; tendința și sezonalitatea la fel.

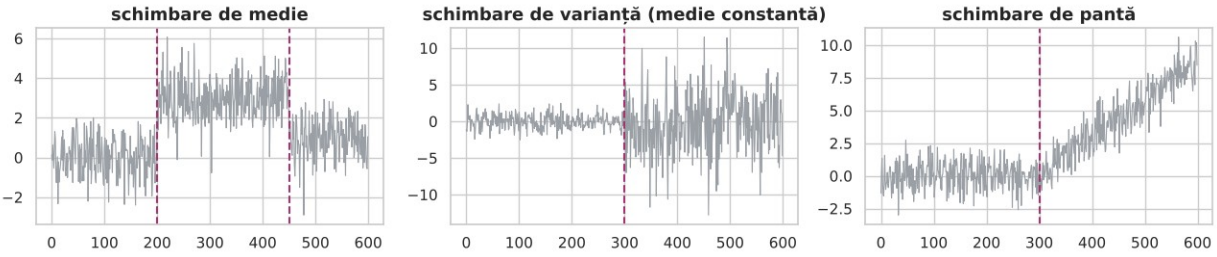


Figura 30. Trei semnale sintetice cu puncte de schimbare cunoscute: o schimbare de medie (la 200 și 450), o schimbare de varianță cu media constantă (la 300), o schimbare de pantă (la 300)

6.2 CUSUM: cel mai simplu detector și cei doi parametri ai lui

O singură schimbare, offline. Suma cumulată a abaterilor de la media globală, `S_k = Σ(x_i − medie)`, are forma de V sau Λ, cu vârful la schimbare: estimarea este `argmax |S_k|`. **CUSUM online** acumulează abaterea față de o medie de referință μ_0 minus o **deriva** `k` care face să dispară zgomotul mic: $g^+_t = \max(0, g^+_{t-1} + (x_t - \mu_0)/\sigma - k)$; alarma se declanșează când $g^+ > h$. Ambii parametri sunt în unități de σ: `k` (toleranța, de obicei **jumătate din saltul pe care vreți să-l detectați**) și `h` (pragul, de obicei 4–5): un `h` mai mare dă mai puține alarme false, dar o întârziere mai lungă.

REZULTAT

```
estimare offline argmax|S|: 299 (adevăr 300)
h               1             3             5              8
k
0.25  alarmă falsă   alarmă falsă   alarmă falsă    întârziere 3
0.75  alarmă falsă   alarmă falsă   întârziere 7   întârziere 11
1.50  alarmă falsă  întârziere 15  întârziere 73  întârziere 123
```

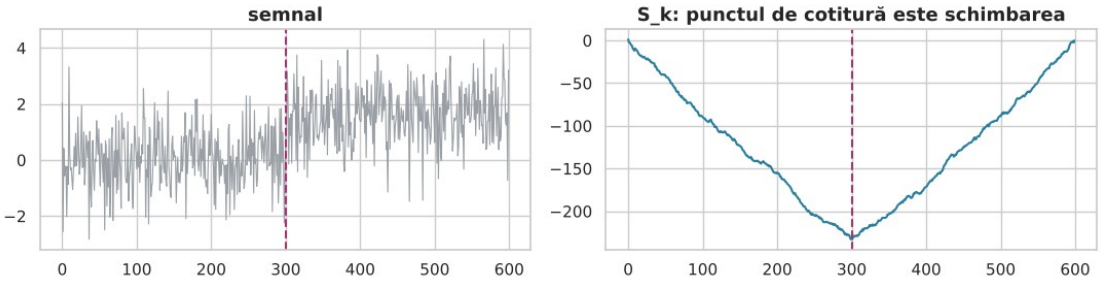


Figura 31. Un salt de 1.5σ la eșantionul 300 și suma lui cumulată: punctul de cotitură este schimbarea

CE ÎNVĂȚĂM

Un `h` mic (1) dă alarme false înainte de schimbare; un `h` mare întârzie alarma; o toleranță `k` cât tot saltul face detectorul lent (întârzieri de 73–123 de eșantioane), pentru că dovada crește atunci cu doar `salt − k` pe eșantion. Un proiect tipic este `k = δ/2`, `h = 4–5`.

6.3 Detecție offline cu ruptures: cost, metodă de căutare și penalitate

Biblioteca `ruptures` separă trei alegeri. **Costul** spune ce se schimbă: `"l2"` (media), `"normal"` (media și varianța), `"rbf"` (orice schimbare de distribuție), `"ar"`. **Metoda de căutare**: `Pelt` (exactă, număr necunoscut de schimbări, folosește `pen`), `Binseg` (lacomă, `n_bkps` sau `pen`), `BottomUp`, `Window(width)`. **Penalitatea** `pen` este prețul pe punct de schimbare și cel mai important parametru: este ceea ce un segment trebuie să plătească pentru a exista. Un bun început este regula BIC, **`pen ≈ 2·σ²·ln n`** pentru costul `l2`, cu σ estimat robust din primele diferențe, `σ = MAD(diff x) / (√2 · 0.6745)`. Apoi verificați **baleierea penalității**: numărul de puncte de schimbare ar trebui să aibă un **palier** larg.

```
def cps(v, pen, model="l2", min_size=2, jump=1):
    return rpt.Pelt(model=model, min_size=min_size,
jump=jump).fit(v).predict(pen=pen)[:-1]
```

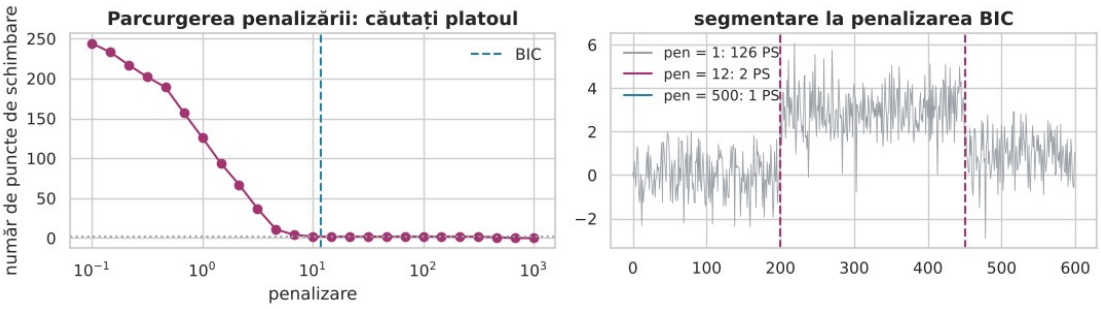


Figura 32. Stânga: numărul de puncte de schimbare în funcție de penalitate are un palier la valoarea adevărată (2). Dreapta: segmentarea la penalitatea BIC

REZULTAT

```
sigma_hat = 0.97   BIC penalty = 11.9
penalizări 0.1 / 1 / BIC / 100 / 1000 -> [244, 126, 2, 2, 0] puncte de schimbare; găsite:
[200, 4…
```

`predict` returnează *indexul de sfârșit al fiecărui segment*, deci ultimul element este întotdeauna `n`; punctele de schimbare sunt celelalte. `min_size` interzice segmentele mai scurte; `jump = 5` evaluează doar fiecare a cincea poziție (de 5 ori mai rapid, eroare de locație de până la 5 eșantioane).

6.4 Costul trebuie să se potrivească cu schimbarea

Semnalul cu schimbare de varianță are **media constantă**: costul `l2`, care presupune varianță constantă, raportează schimbări de medie false în jumătatea zgomotoasă, în timp ce `normal` și `rbf` găsesc singura schimbare adevărată. O **schimbare de pantă** nu este o schimbare de medie: `l2` aproximează rampa cu o scară; folosiți costul liniar pe porțiuni (`model="linear"`, prima coloană = semnalul, celelalte = regresori) – sau lucrați pe prima diferență netezită, ca în exemplul cu biroul de mai jos.

REZULTAT

```
schimbare de varianță, schimbarea adevărată la 300
  l2 (doar media)    : [318, 332, 334, 343, 345, 378, 381, 383, 439, 441, 446, 452, 459,
461, 474…
  normal             : [300]
  rbf                : [300]
  Binseg n_bkps=1, normal: [300]
schimbare de pantă (adevărat 300): l2 -> 7 trepte (o scară care aproximează rampa);  cost
liniar …
```

6.5 min_size, jump și marja de evaluare

`min_size` este cel mai scurt segment în care credeți. O explozie **mai scurtă decât `min_size` nu poate fi delimitată exact**: detectorul o observă totuși, dar trebuie să lărgească segmentul. Ca să **evaluați** un detector, numărați o predicție ca fiind corectă când se află într-o **marjă** față de o schimbare adevărată (`ruptures.metrics.precision_recall`); marja este o alegere de proiectare legată de aplicație: secunde pentru o alarmă, zile pentru un regim.

REZULTAT

```
min_size  2: puncte de schimbare -> [250, 256]
min_size  5: puncte de schimbare -> [250, 256]
min_size 10: puncte de schimbare -> [246, 256]
min_size 20: puncte de schimbare -> [239, 260]
marja  1: precizie 1.00  recall 1.00
marja  5: precizie 1.00  recall 1.00
marja 20: precizie 1.00  recall 1.00
```

6.6 Curățați mai întâi: valorile aberante creează puncte de schimbare false

Un singur vârf de electrod este un „segment” de un eșantion: cu `min_size = 1` detectorul îl raportează ca două puncte de schimbare. Curățarea cu o regulă robustă (Hampel, Capitolul 4) elimină schimbările false și le păstrează pe cele două adevărate; un `min_size` mai mare este o alternativă, dar încețoșează și evenimentele scurte reale.

REZULTAT

```
cu vârfuri, min_size 1 : [80, 81, 200, 320, 321, 450, 520, 521]
cu vârfuri, min_size 5 : [77, 82, 200, 318, 323, 450, 516, 521]
după Hampel + interpolare, min_size 1: [200, 450]
```

6.7 Date reale: EEG și biroul

EEG. Eticheta `classification` se schimbă de patru ori (rândurile 448, 933, 1 748, 2 863): un adevăr de referință aproximativ. Proiectare: log10 → **mediana fiecărui bloc de 30 s** (robustă, iar blocurile sunt aproape independente, deci formula de zgomot alb pentru σ este valabilă; cu un netezitor *mobil* primele diferențe devin

minuscule, σ este subestimat, iar penalitatea BIC mult prea mică – am obținut 43 de schimbări false) → `Pelt` cu `l2` și *dublul* penalității BIC (zgomotul real nu este alb). **Biroul.** CO_2 nu sare când sosesc oameni, ci *crește în rampă*: un model de schimbare de medie pe nivel dă o scară, așa că folosim mediane pe blocuri de 15 minute și apoi **prima diferență** (o rampă devine o schimbare de medie).

REZULTAT

```
EEG  schimbări reale ale etichetei: [448, 933, 1748, 2863]
     detectate (secunde)         : [210, 420, 840, 2340, 2850]
     precizie 0.60 recall 0.75 cu o marjă de 150 s

Birou  începuturi/sfârșituri ale ocupării (index minut): [407, 1065, 1909, 2505, 3288,
3945, 4727…
       detectate din CO2                    : [420, 600, 780, 1050, 1230, 1935, 2115,
2490, 2670,…
        precizie 0.42 recall 1.00 cu o marjă de 90 min (detecțiile în plus sunt pauza de
prânz)
```

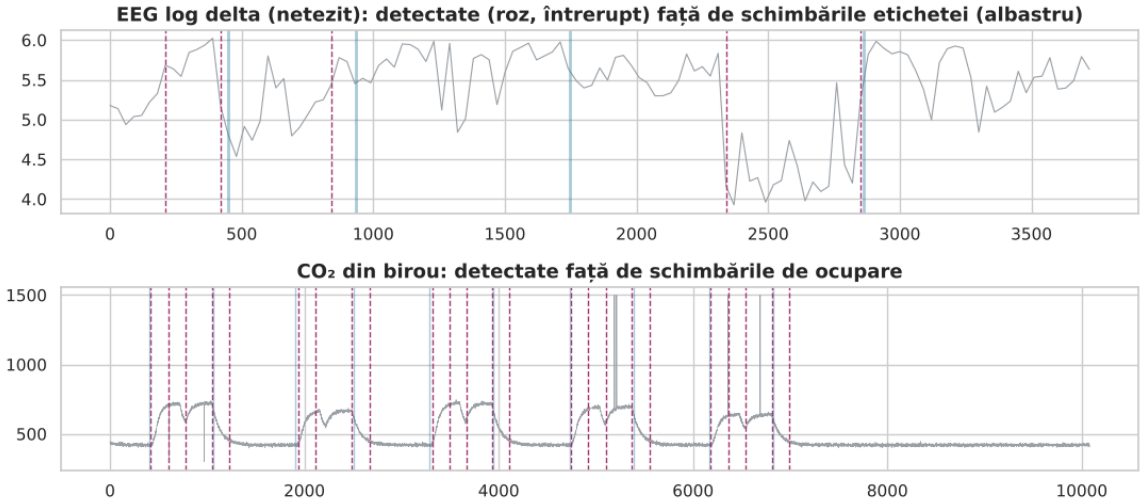


Figura 33. Sus: puterea delta EEG (log), detecțiile (întrerupt) și schimbările de etichetă (albastru). Jos: CO_2 din birou, detecțiile și începuturile/sfârșiturile ocupării

CE ÎNVĂȚĂM

Penalitatea (sau `h` la CUSUM) este butonul alarmelor false: alegeți-o cu un palier/baleiere și o estimare a zgomotului. Costul trebuie să se potrivească cu ce se schimbă; curățați întâi valorile aberante, tendința și nivelul; evaluați cu o marjă care are sens în aplicație. O etichetă precum „somnolent” nu este neapărat o schimbare *statistică* a unui singur semnal: detectorul găsește schimbările care există în date, iar voi trebuie să spuneți care coincid cu eticheta (aici trei din patru).

7 Un pipeline complet de pre-procesare

7.1 Ordinea pașilor

Metodele capitolelor anterioare nu sunt independente: **ordinea** contează, iar fiecare pas are parametri care trebuie notați și justificați. O ordine fiabilă pentru o serie de timp de senzor este:

#	Pas	De ce aici
1	**Grilă regulată** – reindexare la perioada de eșantionare	face golurile vizibile; toți pașii următori presupun eșantioane regulate
2	**Reguli fizice** – valorile din afara domeniului devin `NaN`	cunoștințe de domeniu înaintea statisticii
3	**Valori aberante** – Hampel; detecțiile devin `NaN`	un filtru ar împrăștia vârfurile (secțiunea următoare)
4	**Interpolare** – doar goluri scurte (`limit`)	golurile lungi rămân lipsă și sunt *marcate*
5	**Eliminarea zgomotului** – trece-jos, median, Savitzky–Golay	doar pe date curate și complete
6	**Eliminarea tendinței / sezonalității**	doar dacă pasul următor cere staționaritate
7	**Reducerea dimensionalității, trăsături**	scaler și PCA potrivite doar pe partea de antrenare
8	**Puncte de schimbare, modelare**	scopul

7.2 De ce eliminăm zgomotul după valorile aberante

Un filtru trece-jos împrăștie fiecare vârf peste vecinii săi: energia vârfului devine o umflătură netedă pe care niciun detector nu o recunoaște ca aberantă. Testăm pe o copie curățată a CO_2 din birou în care plantăm zece vârfuri de ±400 ppm, astfel încât adevărul să fie cunoscut.

REZULTAT

```
RMSE față de adevărul filtrat   fără curățare:   5.35   filtru→Hampel:   5.35
Hampel→filtru:   …
```

CE ÎNVĂȚĂM

Întâi filtru, apoi Hampel: RMSE 5.35 – exact la fel ca fără nicio curățare, vârfurile supraviețuiesc sub formă de umflături. Întâi Hampel, apoi filtru: RMSE 0.13.

7.3 O funcție de pipeline cu jurnal de decizii

Fiecare decizie devine un *parametru* al unei singure funcții, iar fiecare efect un *număr* într-un raport: câte valori au fost eliminate, câte interpolate, câte rămân lipsă. Aceasta este ceea ce predați și apărați.

```
clean_co2, rep = preprocess(off.co2_ppm, "1min", lo=300, hi=1500,
                            hampel_window=15, hampel_sigma=5, floor=8, max_gap=60)
```

REZULTAT

```
lipsă_la_început                31
eliminate_de_limitele_fizice     6
eliminate_de_Hampel              0
interpolate                     37
încă_lipsă (gol > max_gap)       0
```



Figura 34. CO_2 din birou înainte și după pipeline: șase defecțiuni eliminate, o întrerupere de 31 de minute interpolată

Tabelul de justificare al raportului vostru are un rând pe pas și trei coloane pe care un cititor le poate verifica:

Pas	Parametri și valoare	De ce această valoare (dovada)
limite fizice	300–1 500 ppm	CO_2 exterior ≈ 420 ppm; > 2 000 nu este credibil cu cel mult 12 persoane
Hampel	fereastră 15, 5σ, prag minim 8 ppm	vârfurile durează 1–2 eșantioane; zgomotul senzorului este de câțiva ppm; pragul 3 a dat 14 alarme false
interpolare	liniară în timp, `limit` 60 min	întreruperea de 31 de minute este mai scurtă decât scara de timp a variațiilor CO_2 (ore)
trece-jos (opțional)	Butterworth, ordin 4, 0.1 cicluri/min	elimină zgomotul peste scara de 10 minute; fază zero pentru că analiza este offline

7.4 Același pipeline pe datele de trafic

Alte date, alți parametri: eșantionare orară, sezonalitate săptămânală și un detector robust care **nu** trebuie să marcheze extremele reale din orele de vârf. Pentru `traffic_volume` din 2017 pasul Hampel (fereastră 25, 6σ, prag minim 300 de vehicule) nu elimină nimic, ceea ce este rezultatul corect; cele patru ore care rămân lipsă aparțin unor goluri mai lungi decât `max_gap = 6`.

REZULTAT

```
lipsă_la_început                47
eliminate_de_limitele_fizice     0
eliminate_de_Hampel              0
interpolate                     43
încă_lipsă (gol > max_gap)       4
cel mai lung gol rămas (h): 3
```

CE ÎNVĂȚĂM

Un pipeline este un **lanț de decizii**. Ordinea este fixată de logică (nu puteți netezi înainte de a scoate valorile aberante, nici detecta puncte de schimbare înainte de a trata nivelul și sezonalitatea); parametrii, de dovezi (spectru, lungimi de goluri, teste cu valori aberante plantate, mascare, validare); iar totul este înregistrat.

8 Rezumat și liste de verificare

8.1 Parametrii dintr-o privire

Metodă	Parametru	Prea mic	Prea mare	Cum se alege
Butterworth	`fc`	elimină semnalul dorit	lasă zgomotul	între dorit și nedorit, verificați răspunsul
	`N`	pantă lentă	întârziere, instabilitate	cel mai mic `N` din `buttord`; folosiți `sos`
Notch	`Q`	larg, consumă vecinii	îngust, ratează deriva interferenței	`f0 / lățime de bandă`
Median	`kernel_size`	vârfurile supraviețuiesc	aplatizează maximele	impar, > 2 × lățimea vârfului
Savitzky–Golay	`window`, `polyorder`	zgomotul rămâne / urmărește zgomotul	aplatizează maximele	fereastra ≈ cea mai îngustă trăsătură; ordin 2–4
Tendință polinomială	grad	tendință rămasă	absoarbe semnalul, explodează la capăt	eroare pe date reținute; cel mai simplu în limita a 5 %
Medie mobilă	fereastră	urmărește zgomotul	pârtinire la curbe	multiplu al perioadei
Diferențiere	lag, ordin	tendință rămasă	zgomot ×2 pe pas	test ADF; lag = perioada
Trece-sus	`fc`	tendință rămasă	atacă ciclul dorit	sub cea mai joasă frecvență dorită; verificați câștigul
Interpolare	metodă, `limit`	—	umple ce nu se poate ști	gol < ¼ perioadă; test de mascare
z-score / IQR	`k`, `c`	alarme false	omisiuni	valori aberante plantate; reguli robuste dacă e contaminat
Hampel	fereastră, `n_sigmas`, prag minim	urmărește vârful	nu vede nivelul local	2 × vârf + 1 … nivel constant; prag ≈ zgomotul senzorului
Isolation Forest	`contamination`	omisiuni	marchează date bune	cunoștințe de domeniu; inspectați fracțiunea
DBSCAN	`eps`, `min_samples`	totul e zgomot	un singur cluster	cotul curbei distanțelor la al k-lea vecin, date scalate
PCA	`k`	structură pierdută	păstrează zgomot	90 %, scree, Kaiser, analiză paralelă, reconstrucție
t-SNE	`perplexity`	insule	pată	5–50, < n/3; doar pentru privit
CUSUM	`k`, `h`	alarme false	întârziere	`k = δ/2`, `h = 4–5`
PELT	`pen`	prea multe schimbări	schimbări ratate	BIC $2\sigma^2 \ln n$, palier în baleiere
	`min_size`	segmente scurte false	începoșează evenimentele scurte	cel mai scurt eveniment în care credeți

8.2 Întrebări înaintea fiecărui pas

- Ce vreau să **păstrez** și ce vreau să **elimin**, și în ce unități (Hz, ore, eșantioane, σ)?
- Care **singur număr** controlează compromisul și ce se întâmplă de o parte și de alta a lui?
- Ce **dovadă** îl alege: un spectru, o eroare pe date reținute, un palier, un test cu valori aberante plantate, un test de mascare?

- Ce face pasul cu pasul **următor** (întârziere, eșantioane pierdute, zgomot dublat, puncte de schimbare false)?
- Este evaluarea **cinstită**: potrivită doar pe partea de antrenare, fold-uri contigue pentru serii de timp, o linie de bază lângă fiecare scor?
- Am **înregistrat** ce a eliminat sau modificat pasul?

8.3 Lecturi suplimentare

- SciPy – procesarea semnalelor (butter, buttord, firwin, savgol_filter, medfilt, iirnotch)
- ruptures – detecția punctelor de schimbare în Python
- statsmodels – descompunerea STL
- scikit-learn – descompunere (PCA) și detecția valorilor aberante
- scikit-learn – detecția noutăților și a valorilor aberante

9 Tema

9.1 Caietul de exerciții

Deschideți `Lab4_Exercitii.ipynb`. Are 51 de exerciții în șapte părți. În fiecare exercițiu *alegeți un parametru pe baza dovezilor* și apoi explicați alegerea în celula „Interpretarea voastră”. Rulați exercițiile în ordine; un traceback roșu într-o autoverificare înseamnă doar că exercițiul nu este încă rezolvat.

Partea	Exerciții	Partea sintetică (adevăr ascuns)	Partea cu date reale
1 Filtre	11	citirea spectrului, cele patru tipuri de filtre și ordinul dintr-o specificație, media mobilă pentru 50 Hz, Butterworth dintr-o specificație, întârzierea filtrării cauzale, lungimea FIR, Q la notch, nucleul medianului, fereastra Savitzky–Golay	benzi de trafic; vârfuri de CO_2
2 Eliminarea tendinței	7	gradul polinomului, fereastra mobilă, costul diferențierii, frecvența de tăiere a trece-sus, testul ADF	lag sezonier și STL pe trafic; linia de bază EEG
3 Interpolare	7	cea mai bună metodă pe lungime de gol, cel mai lung gol, depășire, marcaje neregulate	recensământul golurilor și testul de mascare pe trafic; golul de 31 de minute din birou
4 Valori aberante	8	pragul z, mascare, date asimetrice, fereastra Hampel, contaminare, eps la DBSCAN și scalare	CO_2 și temperatura din birou; fișierul de trafic
5 Dimensionalitate	7	scalare, numărul de componente, reconstrucție, praguri de selecție, scurgerea de informație, perplexity la t-SNE	EEG: PCA și evaluare cinstită
6 Puncte de schimbare	9	CUSUM, penalitate și palier, cost, `min_size`, marjă, curățare	puterea delta EEG; CO_2 din birou
7 Pipeline	2 + proiect	—	alegeți parametrii lui `preprocess` pentru CO_2 și temperatură; mini-proiect liber

> **CUM FUNCȚIONEAZĂ AUTOVERIFICĂRILE**
> Numerele sunt comparate **după rotunjire la 2 zecimale**, iar valorile așteptate sunt stocate ca amprente, deci răspunsurile nu pot fi citite în caiet: trebuie calculate. Unele verificări sunt condiții (✅ când rezultatul are proprietatea așteptată, de exemplu „splineul cubic depășește cu mai mult de 1”). Caietul folosește doar `numpy`, `pandas`, `scipy`, `statsmodels`, `scikit-learn` și `ruptures` (`pip install ruptures`).

9.2 Mini-proiectul (Partea 7.3)

Alegeți **un singur** set de date – trafic (`traffic_volume`, `temp`, `rain_1h`) sau EEG (`attention`, `meditation` și puterile pe benzi) – și scrieți un raport de o pagină plus caietul. Trebuie să conțină: **(1)** diagnosticul (goluri, valori aberante, spectru/sezonalitate, tendință), cu câte o figură; **(2)** pipeline-ul cu tabelul lui de justificare (pas – parametri – dovadă); **(3)** o comparație înainte/după pe o **sarcină din aval** (de exemplu F1 la detectarea schimbărilor de etichetă sau RMSE al unei predicții pentru ora următoare), astfel încât pre-procesarea să fie judecată după efectul ei; **(4)** două limitări ale pipeline-ului vostru.

Criteriu	Pondere
Diagnostic cu figuri	20 %
Alegerea parametrilor pe bază de dovezi	30 %
Comparația înainte/după pe o sarcină din aval	30 %
Claritate, înregistrarea deciziilor, limitări	20 %

9.3 Predare

Predați `Lab4_Exercitii.ipynb` executat (*Kernel → Restart & Run All*, toate verificările ✅, răspunsurile scrise și graficele prezente) și raportul mini-proiectului (PDF).